Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Automatic Control Laboratory

Semester Project

Deep reinforcement learning for Flexible Job Shop Scheduling

Alberto Fasulo
April 2026

Advisors
Yu Tang
Muhammad Zakwan
Efe Balta
Prof. John Lygeros

Chapter 1

Abstract

The Flexible Job Shop Scheduling Problem (FJSSP) is one of the most practically relevant variants of production scheduling and is NP-hard already in its non-flexible form. In real manufacturing environments it must be solved under continuous stochastic disruptions: machines fail without warning, new customer orders arrive while the shop floor is active, and existing orders are occasionally cancelled. The two classical solution families are unable to cope with this regime. Exact methods and meta-heuristics produce strong offline solutions but require expensive re-optimisation whenever a disruption invalidates the current plan, and they are therefore unsuitable for real-time decision making. Priority dispatching rules (PDRs) are fast and reactive but no single rule dominates across operating regimes: the best rule when the shop is congested is generally not the best rule when a machine has just failed or when a burst of new orders is arriving. This motivates learning a controller that selects the sequencing and routing rules according to the current shop-floor state.

This thesis develops such a controller. The FJSSP rule-selection problem is cast as a Markov decision process with a 51-dimensional observation vector that summarises the global machine load, per-machine queue and workload features, aggregate job statistics, the type of the current event, and a forward-looking *arrival pressure* signal that encodes the expected intensity of future dynamic disruptions. The action space is the Cartesian product of five sequencing rules (SPT, LPT, MWKR, MOR, FIFO) and five routing rules (MinWL+P, MinQueue, SPT-Rt, BalUtil, EarliestFree), yielding 25 joint actions. Two architectural variants are studied: a single-head *Discrete(25)* formulation, in which the policy learns a fully joint distribution over rule pairs and can therefore capture synergistic combinations, and a two-head *MultiDiscrete([5,5])* formulation, in which sequencing and routing are factorised into two independent categorical heads sharing a common feature extractor. Both formulations are trained with Proximal Policy Optimisation (PPO).

The main contribution is a three-phase *progressive curriculum along the disruption-intensity axis*. To the best of our knowledge this is the first curriculum-learning strategy of its kind for dynamic FJSP: prior curriculum work in scheduling addresses static instances and varies problem scale or instance difficulty, rather than disruption intensity. The proposed curriculum begins with eight fully static environments, advances to a medium-intensity domain-randomised regime that introduces all three disruption types at reduced rates, and concludes under the full disruption regime. A dedicated callback replaces the training environment only at the end of a completed PPO rollout buffer, so every gradient update operates on a homogeneous batch of on-policy experience drawn from a single curriculum phase. This design avoids the gradient-conflict pathology observed when static and dynamic episodes are mixed within the same rollout buffer.

On the small instance (6 machines, 20 initial jobs) the curriculum-trained PPO agent wins five of six disruption scenarios with statistical significance and ties the sixth (Only breakdowns, $p = 0.16$), with improvements of up to 4.8% on arrival-driven configurations. The same training

strategy transfers cleanly to a larger instance (8 machines, up to 40 jobs), where it wins all six scenarios with improvements of up to 4.8% and, in fact, produces *stronger* relative gains than on the small instance, consistent with the intuition that adaptive rule selection has more room to exploit in more complex problems. A dedicated generalisation study trains a curriculum model on a fixed pool of only ten job instances while continually varying the disruption realisation, and shows that the policy transfers to job structures it never saw during training with Δ gaps within $\pm 2\%$ across all scenarios, providing evidence that the agent learns a genuine adaptive scheduling strategy rather than instance-specific tricks.

The role of the curriculum itself is isolated by a controlled 2×2 factorial ablation that crosses curriculum-versus-no-curriculum against fixed-versus-random job pools. The ablation results indicate that the curriculum is the main factor: removing it degrades scenario coverage from 6/6 point-estimate wins (5/6 statistically significant) to 3/6, irrespective of job-pool fixation. A second ablation over phase counts shows that the three-phase design outperforms both two-phase (which suffers from a single abrupt difficulty jump) and four-phase (which fragments the timestep budget without adding useful difficulty progression) variants in the evaluated scenarios. A direct algorithm-level comparison applies the same three-phase curriculum to a DQN agent under matched hyperparameters and shows that DQN wins only 3/6 scenarios. We attribute this gap to the off-policy replay buffer, which retains stale transitions from earlier curriculum phases that contaminate the Q-value targets in subsequent phases, whereas PPO discards all experiences at each rollout boundary and trains exclusively on data aligned with the current phase.

A multi-seed validation retrains the two strongest models under an independent weight initialisation (seed 123 versus the primary seed 42) and confirms that Model B achieves 5/6 statistically significant wins on unseen job structures under seed 42 (Only breakdowns is a statistical tie, $p = 0.16$), with consistent point-estimate performance under both initialisations, and that the curriculum advantage over no-curriculum models is consistent at 3–5 percentage points across both seeds.

An additional study evaluates robustness to non-memoryless failure distributions using a Weibull failure model ($k = 2$) and an extended 57-dimensional observation with per-machine age features. A model trained directly in this environment improves over the Exponential-trained Model D by 0.6%–3.9% across the six Weibull scenarios, while Model D itself still beats the static rules in all scenarios.

Finally, a negative result is reported: a flat MLP policy trained over variable machine counts (15- and 20-machine input slots, using a disabled-machine encoding) fails to outperform any static rule. The failure is shown to be a structural property of the MLP representation, which has no inductive bias for the natural permutation symmetry of machines, rather than a failure of curriculum learning or PPO. This finding motivates future work on graph- or attention-based architectures that can amortise across instance sizes by construction.

Chapter 2

Preface

This work was carried out as a Semester Project at the Automatic Control Laboratory (IFA), ETH Zurich, between March and September 2025. The project falls under the laboratory's ongoing research line on data-driven control and learning for smart-manufacturing systems, and addresses the Flexible Job Shop Scheduling Problem (FJSSP) under dynamic conditions characteristic of real industrial plants.

I am grateful to my supervisors, Yu Tang (ZHAW), Muhammad Zakwan (Inspire AG), and Efe Balta (Inspire AG), for their continuous guidance, the regular technical discussions, and the maintained reinforcement learning environment that served as the starting point of this thesis. I would also like to thank Prof. John Lygeros for hosting the project at IFA and for the broader research context within which this work was conducted.

Some results presented here are intended to feed back into the laboratory's research line on learning-based scheduling. Any errors or omissions remain my own.

Alberto Fasulo *Zurich, April 2026*

Contents

1 Abstract **3**

2 Preface **5**

List of Figures **11**

List of Tables **13**

3 Introduction **1**
3.1 Contributions 2
3.2 Outline 3

4 Related Work **5**
4.1 DRL for Static FJSP 5
4.2 DRL for Dynamic FJSP 5
4.3 Curriculum Learning for Scheduling 6
4.4 Research Gap 7

5 Problem Definition **9**
5.1 Formal Description 9
5.2 Dynamic Extensions 10
5.3 Instance Configuration 10

6 State, Action, and System Pipeline **13**
6.1 System Pipeline 13
6.2 Observation Vector (51 Dimensions) 14
6.3 Action Space 17
6.4 Decision Points and State Transitions 19

7 Algorithm and Model Architecture **21**
7.1 Proximal Policy Optimisation 21
7.2 Network Architecture 22
7.3 Single-Head Formulation (Discrete(25)) 23
7.4 Two-Head Formulation (MultiDiscrete($[5, 5]$)) 23
7.5 Comparison of the Two Formulations 25

8 Training Procedure **27**
8.1 Dynamic Episode Horizon and Reward Normalisation 27
8.2 Instance Generation, Dataset Split, and Generalisation 27
8.3 Reproducibility and Seed Strategy 28
8.4 Hyperparameters 29

8.5 Reward Function . . . 30
8.6 Training Algorithm . . . 31

9 Event-Based Scheduling **33**
9.1 Event Priority Queue . . . 33
9.2 Stochastic Event Generation . . . 34
9.3 Stale Event Detection . . . 35
9.4 Breakdown and Re-routing Procedure . . . 35

10 Implementation Details **37**
10.1 Module Structure . . . 37
10.2 Key Data Structures . . . 37

11 Baseline Results and Policy Analysis **39**
11.1 Evaluation Protocol . . . 39
11.2 Quantitative Results . . . 39
11.3 Context-Sensitive Policy Behaviour . . . 40
11.4 Policy Interpretability . . . 41
11.5 Schedule Visualisation . . . 41

12 Disruption Robustness Ablation (Zero-Shot) **43**
12.1 Motivation and Protocol . . . 43
12.2 Scenarios . . . 44
12.3 Results . . . 44
12.4 Analysis . . . 44
12.5 Identified Limitation and Proposed Fix . . . 45

13 Curriculum Learning and Experimental Evaluation **47**
13.1 Approach and Experimental Setup . . . 47
13.1.1 Problem Setting Recap . . . 47
13.1.2 Training Strategies Considered . . . 47
13.1.3 Baseline Comparison . . . 48
13.2 Results: Dynamic-Only Training . . . 48
13.2.1 Ablation Study Across Scenarios . . . 49
13.3 Results: Mixed Training (6 Dynamic + 2 Static) . . . 49
13.4 Curriculum Learning in Scheduling: Prior Art and Our Rationale . . . 50
13.5 Results: Progressive Curriculum Training . . . 52
13.6 Convergence Study: 10 M Step Training . . . 53
13.7 Instance-Size Generalisation: Large Instance Evaluation . . . 54
13.8 Scaling to Variable Instance Sizes: Limits of the MLP Architecture . . . 55
13.9 Generalisation Study: Fixed Job Pool with 16 Event Realisations . . . 56
13.10 Four-Model Comparison: Curriculum vs. No Curriculum × Random vs. Fixed Jobs 57
13.10.1 Statistical Significance of Model B and Model D . . . 58
13.11 Multi-Seed Validation . . . 59
13.12 Curriculum Phase-Count Ablation . . . 61
13.13 DQN vs. PPO: Algorithm Comparison . . . 62
13.14 Comparison with Existing DRL Methods . . . 63
13.15 Robustness to Non-Memoryless Failure Distributions . . . 64
13.16 Extra Baselines: Oracle, Random, and Event-Type Switchers . . . 65

14 Discussion and Conclusion **69**
14.1 Discussion 69
14.2 Conclusion 71
14.3 Future Work 72

Bibliography **73**

List of Figures

6.1 Complete system pipeline from episode initialisation to metric logging. The agent is queried at every decision point produced by the event loop. 14

6.2 Structure of the 51-dimensional observation vector. Proportional widths in the bar chart reflect the number of dimensions in each block. 15

6.3 The 25-element joint action space. Each cell shows action index a. Shaded cells lie on the diagonal where $s = r$. 19

7.1 MLP architecture. Policy and value heads share the feature extractor (256–256–128, Tanh activations). Tanh keeps activations bounded, consistent with the $[0, 1]$-normalised inputs. 23

7.2 Two-head MLP architecture. The shared trunk is identical to the single-head variant; the output layer is replaced by two Dense-5 heads with independent softmax normalisation. 24

9.1 Event types and their consequences. Every event ultimately triggers a call to the sequencing and routing rule applicator. 34

11.1 Three-panel Gantt chart (seed 42). **Top:** PPO agent (deterministic), $C_{\max} = 149.49$. **Middle:** best static rule pair (MWKR + MinWorkload+Proc), $C_{\max} = 154.21$. **Bottom:** random baseline, $C_{\max} = 192.24$. Each coloured bar is one operation; colours are consistent across panels so the same job can be tracked across policies. The red dashed vertical line marks the final makespan $C_{\max}$ for each policy. Vertical dotted lines indicate cancellation, arrival, and breakdown events. 42

13.1 Dynamic-only training pipeline. All eight parallel environments apply full domain randomisation at every episode reset; the disruption parameters are never zero. . 48

13.2 Mixed training pipeline $(6 + 2)$. Six dynamic and two static environments contribute to the same rollout buffer each iteration. The opposing gradient signals cause a conflict that degrades performance in both dynamic and static scenarios. 49

13.3 Progressive curriculum training pipeline. The `CurriculumCallback` replaces the training environment at the end of a complete rollout buffer (never mid-rollout), so each PPO update sees a homogeneous batch. Disruption difficulty increases monotonically across the three phases. 52

13.4 Training curves (`eval/mean_reward`) for the four models over 7 M steps. Model A (no curriculum, fixed jobs) achieves the highest reward during training because it is always evaluated on a simple, static-like environment. Curriculum models B and D show lower rewards during training: each phase transition introduces harder disruptions, temporarily degrading the eval score before the policy re-adapts, yet they converge faster to a robust policy once they stabilise. Despite appearing worse during training, curriculum models generalise better in zero-shot evaluation (Table 13.10), confirming that a higher training reward does not imply a stronger final policy. 58

List of Tables

5.1 Environment parameters used throughout all experiments. 11

7.1 Single-head vs. two-head formulation summary. 25

8.1 Training hyperparameters for both configurations. 29

11.1 Evaluation results over 300 independent episodes at 5 M training steps. Lower $C_{\max}$ is better. Baselines are averaged over 300 episodes. Gap is relative to the best static rule pair. 40

11.2 Dominant rule per event type. Two-head PPO (seed 42, 5 M steps), averaged over 300 evaluation episodes. Values in parentheses are marginal probabilities. 40

12.1 Disruption configurations used in the zero-shot ablation. The baseline row matches the training distribution exactly. 44

12.2 Zero-shot disruption robustness. For each scenario the best static rule pair is found by exhaustive sweep; the gap is $(C_{\max}^{\text{PPO}} - C_{\max}^{\text{best-static}})/C_{\max}^{\text{best-static}} \times 100\%$. Negative gap means PPO outperforms the best static rule. 44

13.1 PPO vs. static baselines (dynamic training). 48

13.2 Performance across different disruption scenarios (dynamic-only training). 49

13.3 Mixed training results. 50

13.4 Zero-shot disruption robustness, curriculum-trained two-head PPO (7 M steps). Negative gap means PPO outperforms the best static rule. 53

13.5 Comparison of 7 M and 10 M curriculum runs. Gap is relative to the best static rule. Negative gap = PPO outperforms best static. 53

13.6 Zero-shot disruption robustness on the **large instance** (8 machines, up to 40 jobs, OBS-DIM = 61). All gaps are relative to the best static rule for that scenario. Negative gap = PPO outperforms best static. 54

13.7 Gap vs. best static rule: small (6 machines, OBS-DIM = 51) vs. large (8 machines, OBS-DIM = 61) instance. All values in %; negative = PPO wins. 54

13.8 Variable-size generalisation: gap vs. best static rule (MWKR + MinWL + P) across all tested active-machine counts. Positive gap = PPO *worse* than best static. Compare with Table 13.6 (dedicated 8-machine model, all gaps negative). 55

13.9 Generalisation study: gap vs. best static rule on known and unseen job seeds. Δ = Unseen − Known. Values near zero indicate robust generalisation. 57

13.10 Gap vs. best static rule for all four models across six disruption scenarios (confirmed results from dedicated ablation runs: D from `ablation_disruptions.py`; A and B from `ablation_generalization.py`, unseen job seeds 200+). Negative = PPO outperforms best static. Best result per scenario in **bold**. 59

13.11 Model D (Rand+CL, seed 123) vs. best static rule: paired Wilcoxon signed-rank test ($n = 500$ matched episodes, two-sided). Gap [95 % CI] is the bootstrap CI on the per-episode paired gap $\Delta C_{\max}/C_{\max}^{\text{static}}$. All Bonferroni-corrected p-values are below 0.001. $^{***}p < 0.001$; effect size $r = Z/\sqrt{n}$. 59
13.12 Model B (Fixed+CL, seed 42, unseen job seeds 200+) vs. best static rule: paired Wilcoxon signed-rank test ($n = 500$ matched episodes, two-sided). $^{***}p < 0.001$, $^{**}p < 0.01$, ns = not significant. Effect size $r = Z/\sqrt{n}$. 60
13.13 Multi-seed validation: gap vs. best static rule for Models B and D under two independent training seeds. Model B is evaluated on unseen job seeds (200+) throughout. Negative gap = PPO outperforms best static. All entries are point estimates; paired significance testing was conducted for Model B seed 42 and Model D seed 123 only (Section 13.10.1). 60
13.14 Curriculum phase-count ablation, gap vs. best static rule across six disruption scenarios (fixed job pool, 7 M total steps). Negative gap = PPO outperforms best static. Best result per scenario in **bold**. 61
13.15 DQN vs. PPO (Model B), gap vs. best static rule across six disruption scenarios (same 3-phase curriculum, 7 M steps). Negative gap = agent outperforms best static. Best result per scenario in **bold**. 62
13.16 Comparison of relative improvement over best static dispatching rule. Chang et al. results are taken directly from their paper [8]. Our results are averaged across six disruption scenarios (Model B, curriculum PPO). Direct numerical comparison is not possible due to different simulators and instance generators; both methods are reported relative to their own best static baseline. 64
13.17 Gap vs. best static rule under Weibull failures ($k = 2$). Model D was trained on Exponential failures (OOD test); the Weibull model was trained on Weibull $k = 2$ failures (in-distribution). Δ = Weibull gap − Model D gap; negative Δ indicates the Weibull model outperforms Model D. Negative gap = PPO outperforms best static. 65
13.18 Extra baselines vs. PPO (Model D, seed 123, $N = 300$). Gap % = (method − PPO)/PPO × 100. Positive = method worse. Oracle sig. and switcher sig. from paired Wilcoxon. SwA = self-distilled; SwB = literature-prior. 66
13.19 Extra baselines vs. PPO (Model B, seed 42, unseen job seeds 200+, $N = 300$). Same sign convention as Table 13.18. 66
13.20 Extra baselines vs. PPO (Weibull model, seed 123, Weibull $k = 2$ failures, $N = 300$). SwA = standard-model lookup (Table 11.2 marginals, *not* re-distilled from the Weibull model). 67

Chapter 3

Introduction

Scheduling problems arise across manufacturing, cloud computing, logistics, and service operations: in each setting a finite set of tasks competes for a shared set of resources, and the quality of the allocation has a direct impact on cost, throughput, and service-level performance. Among the many formulations studied in the operations-research and scheduling literature, the *Flexible Job Shop Scheduling Problem* (FJSSP) is one of the most practically relevant and computationally demanding. It extends the classical Job Shop Scheduling Problem (JSP) by permitting each operation to be processed on any machine drawn from a predefined eligible subset, typically with a machine-dependent processing time. This generalisation is faithful to the structure of modern manufacturing cells, in which several machines have overlapping but not identical capabilities, and it improves resource utilisation by allowing the scheduler to balance load across substitutable machines. This expressiveness comes at a computational cost: where the JSP already requires solving an assignment problem with precedence and machine-exclusivity constraints, the FJSSP simultaneously solves the corresponding *routing* problem, which eligible machine each operation should use, and the joint decision space grows exponentially with the number of operations and machines. The problem is NP-hard in its non-flexible form [11], and the flexibility extension makes it strictly harder.

The classical solution toolkit was developed under an implicit assumption of *static* problems: the full set of jobs, their processing times, and the set of available machines are all known at time zero and remain fixed throughout the planning horizon. Real-world job shops, however, are rarely static. Machines fail without warning and require non-deterministic repair times. New customer orders arrive in real time and must be incorporated into the active schedule rather than batched into a future planning cycle. Existing orders are occasionally cancelled mid-production, either at customer request or as a consequence of an upstream quality issue. Each of these events invalidates assumptions on which a previously computed schedule was based. Exact methods such as branch-and-bound or mixed-integer linear programming are doubly handicapped under these conditions: they are computationally intractable already at realistic instance sizes, and they are entirely rigid once the schedule is computed; any disruption requires a full re-solve, which is prohibitively expensive at the millisecond-to-second timescales required by a manufacturing execution system. Meta-heuristics, genetic algorithms, simulated annealing, tabu search [6], relax the optimality target in exchange for tractability and produce strong offline solutions on instances of practical size, but they share the same re-optimisation cost when the problem changes, and they are not inherently online algorithms.

Priority dispatching rules (PDRs) occupy the opposite end of the spectrum and have therefore become the de-facto choice for online scheduling in industry. A PDR is a deterministic function that, when a machine becomes idle, selects the next job from a non-empty queue according to a simple scoring criterion: shortest remaining processing time (SPT), most work remaining over all subsequent operations (MWKR), first-in-first-out (FIFO), and so on. Classical rules of

this kind [5, 22] are fast: their computation is essentially free at any practical instance size, they are inherently online, and they are trivial to implement on a manufacturing execution system, which is precisely why they remain the workhorses of real-time shop-floor control [10]. Their fundamental weakness is the absence of any context-dependent adaptation: each rule encodes a single heuristic priority, and *no single rule dominates across all shop configurations and disruption patterns* [12]. SPT, for example, excels when queues are long because clearing short jobs frees machine capacity quickly, but it starves long jobs and degrades sharply when those long jobs are on the critical path. MWKR is preferable when a bottleneck machine governs throughput because it prioritises jobs that still have the most work to do, but it provides no benefit when no clear bottleneck exists. Committing to one rule for an entire episode is therefore suboptimal, and a substantial body of empirical work [12] confirms that the performance ranking of dispatching rules depends sensitively on instance-level features, such as machine utilisation, job-mix variability, and disruption intensity, that cannot be inferred at design time.
This thesis addresses this trade-off by treating **rule selection itself** as the learning target. Instead of learning a direct job-to-machine assignment policy, which requires the policy to choose, at every step, from an action space whose size scales with the number of pending operations multiplied by the number of eligible machines, the agent is given a fixed catalogue of five sequencing rules and five routing rules and asked to select, at every decision point, the most appropriate *pair* for the current shop-floor state. The resulting action space contains only $5 \times 5 = 25$ entries regardless of instance size, which yields three practical advantages: (i) the policy can be a compact MLP rather than a graph or attention network with millions of parameters, (ii) every action is by construction a feasible scheduling decision (the underlying rules guarantee feasibility and respect precedence and eligibility constraints), and (iii) the policy is directly interpretable: a trained model can be inspected by reading off which sequencing rule it applies under which event type, recovering the kind of human-readable heuristic that practitioners are willing to deploy on a real shop floor. The agent therefore behaves as an *adaptive hyper-heuristic*, in the sense of the hyper-heuristic literature: it does not solve the scheduling problem from scratch, but learns to select among lower-level heuristics whose individual behaviours are well understood.

3.1 Contributions

The contributions of this thesis are the following:

- A complete deep reinforcement learning environment for the dynamic FJSSP with machine breakdowns, dynamic job arrivals, and order cancellations, implemented as a Gymnasium [31] environment and trained with Stable-Baselines3 [25]. The environment uses an event-driven simulation that issues a decision request only when a scheduling choice is actually required, eliminating the wall-clock-tick artefacts of time-stepped simulators and producing a Markov decision process whose horizon is governed by the number of events rather than by simulated time.

- A side-by-side analysis of two PPO [27] formulations, a single-head `Discrete(25)` variant that learns the full joint distribution over rule pairs, and a two-head `MultiDiscrete([5,5])` variant that factorises sequencing and routing into independent categorical heads, with a quantitative analysis of the trade-off between synergistic and factorised rule learning under matched hyperparameters.

- A curriculum-learning strategy, to our knowledge the first defined along the **disruption-intensity axis**: a three-phase progressive curriculum that resolves the gradient-conflict problem observed when static and dynamic episodes are mixed within a single rollout buffer, and

that achieves 5/6 statistically significant scenario wins against the best static rule (Only breakdowns is a statistical tie).

- A controlled 2×2 ablation isolating curriculum learning from job-pool fixation, and a phase-count ablation showing that the three-phase design outperforms the two- and four-phase alternatives in the evaluated scenarios.
- A direct PPO-versus-DQN comparison under an identical curriculum and matched hyperparameters, with an explanation of why off-policy replay buffers are disadvantaged under non-stationary curriculum training.
- A scaling study (6, 8, 15, and 20 machines) that delimits the regime of validity of a flat MLP policy and motivates graph-based or attention-based architectures for size-agnostic scheduling.
- A Weibull failure environment with a machine-age observation feature (OBS_DIM = 57) and a curriculum-trained model evaluated against the Exponential-trained Model D under Weibull failures ($k = 2$). The Weibull model outperforms Model D on all six disruption scenarios (point estimates, Table 13.17). Model D itself beats the best static rule on all six Weibull scenarios (point estimates); on the standard Exponential instance it achieves 6/6 statistically significant wins (paired Wilcoxon; Model B is 5/6 + 1 tie). Together these show that the curriculum-trained policy retains substantial robustness under a shift in the failure-time distribution.

3.2 Outline

Chapter 4 reviews DRL for static and dynamic FJSP and curriculum learning for scheduling, and identifies the research gap addressed by this thesis. Chapter 5 formalises the FJSSP, defines the four event types that constitute the dynamic extensions, and lists the instance parameters used throughout all experiments. Chapter 6 describes the system pipeline that connects the simulation, the agent, and the rule applicator, and details the 51-dimensional observation vector and the 25-action joint action space. Chapter 7 introduces PPO from first principles, motivates its choice over the main alternatives, and presents the two network architectures used for the single-head and two-head formulations. Chapter 8 discusses the training procedure, including the instance-specific reward normalisation that is central to a fair gradient signal across heterogeneous instances, the reward design, and the reproducibility and seed-management protocol. Chapter 9 describes the event-driven scheduling simulation, including the priority-queue mechanics, stochastic event generation, and the stale-event detection logic that keeps the simulation consistent under concurrent breakdowns and dispatch events. Chapter 10 documents the implementation, module structure, and core data classes. Chapter 11 reports the baseline quantitative results and analyses what kind of policy the agent has learned. Chapter 12 presents the zero-shot disruption robustness study, which exposes a generalisation gap in the dynamic-only policy and motivates the curriculum. Chapter 13 is the methodological core of the thesis: it introduces the progressive curriculum, presents the generalisation study, the 2×2 ablation, the phase-count ablation, the DQN comparison, and the scaling study. Chapter 14 discusses limitations, summarises the findings, and outlines future work.

Chapter 4

Related Work

The literature relevant to this thesis spans three communities: deep-reinforcement-learning approaches to static FJSP, DRL approaches to dynamic FJSP, and curriculum learning applied to scheduling. We review each in turn and then identify the precise intersection that this work addresses.

4.1 DRL for Static FJSP

The dominant body of literature formulates the static FJSP as a Markov decision process and trains a DRL agent to make direct job-to-machine assignment decisions at each step. "Direct assignment" means that the action space contains, in some form, the joint choice of the next operation to dispatch and the eligible machine to assign it to. The size of this action space grows with the number of pending operations multiplied by the eligibility set size, so the architectural choice becomes the central design question: how to represent a scheduling state in which both the dimension of the action space and the relevant context vary across decisions. Graph neural networks (GNNs) and attention mechanisms are the standard answer because they naturally represent the disjunctive-graph structure of the scheduling state and produce policies that are by construction invariant to the number and ordering of jobs and machines.

Song et al. [29] propose a heterogeneous GNN that captures complex relationships between operations and machines, achieving competitive results on the Brandimarte benchmark. Wang et al. [32] introduce a Dual Attention Network that combines self-attention over operations and machines with a DRL policy and represents one of the current state-of-the-art direct-assignment methods. Lei et al. [15] propose a multi-pointer graph network with a Multiple-PPO training algorithm. All of these methods train and evaluate on **static** instances where all jobs are known at time zero and no disruption occurs. They also share the property that interpretability is limited: a trained GNN dispatcher implicitly encodes a heuristic over operation–machine pairs, but extracting a human-readable description of that heuristic is non-trivial.

The present work takes a deliberately different stance on both axes. We trade the universality of a direct-assignment policy for the interpretability and computational simplicity of rule selection, and we address dynamic rather than static environments. The trade-off is that our policy cannot synthesise novel scheduling heuristics outside the catalogue of 25 rule pairs, but within that catalogue every action is both feasible by construction and explainable to a human operator.

4.2 DRL for Dynamic FJSP

A growing body of work addresses dynamic or stochastic scheduling with DRL. The space is heterogeneous: different papers use different disruption types, different objectives, different sim-

ulators, and different baseline rules, which makes direct numerical comparison unreliable. Three works are particularly relevant to this thesis.

Liu et al. [18] propose a multi-agent RL framework for dynamic FJSP with two independent agents, a sequencing agent and a routing agent, that make direct job-to-machine assignments. This is the most cited DRL work on dynamic FJSP. However, the objective is *total tardiness* rather than makespan, the simulator is a custom SimPy-based environment, and no curriculum is applied. Direct numerical comparison with our work is therefore methodologically unsound, but the two-agent decomposition foreshadows the sequencing–routing factorisation explored in our two-head architecture.

Chang et al. [8] address dynamic FJSP where machines can break down and resume service, with makespan as the objective. They train a GIN-based PPO Learning-to-Dispatch model and report 1.0%–9.2% improvement over the best priority dispatching rule across six instance sizes. This is the closest existing work to ours in problem formulation and objective. Their approach differs from ours in two important respects: they make direct assignment decisions rather than rule selections, and they do not apply curriculum learning. Section 13.14 provides a detailed quantitative comparison of our results against theirs. A complementary line of work by Chang et al. [9] tackles dynamic FJSP with *random job arrival* and variable processing times using a DDQN-based dispatcher; together with [8] this establishes that DRL-based dynamic-FJSP dispatchers can outperform heuristics and meta-heuristics under a range of disruption types, but without addressing the curriculum question that is the focus of this thesis.

Further related work includes hybrid intelligence via rule selection with a Double Dueling DQN [16] for dynamic JSP (not FJSP), Maskable PPO with Petri-net action masking [2] for dynamic JSP, and DQN with prioritised experience replay [1] for dynamic FJSP in reconfigurable manufacturing cells. None of these works applies curriculum learning, and several of them are confined to non-flexible JSP or to specialised manufacturing cell variants.

4.3 Curriculum Learning for Scheduling

Curriculum learning [4] trains a model on a sequence of tasks ordered from easy to difficult. The intuition is that early exposure to simple instances lets the model build a strong foundation that subsequent exposure to harder instances can extend, analogously to how human learners benefit from a structured educational sequence. A substantial survey of curriculum methods for general RL is given by Narvekar et al. [21]. The application to scheduling has been explored by a small but growing body of work, all of which targets **static** instances and varies either **problem scale** or instance difficulty as the curriculum dimension.

Iklassov et al. [13] were the first to apply curriculum learning to DRL-based dispatching rules for static JSP, showing that training on small instances (6×6 jobs/machines) before large instances (30×20) significantly improves generalisation to large unseen instances. The mechanism they identify is reward density: small instances produce shorter episodes with more frequent terminal rewards, accelerating early learning and providing a strong inductive bias before the policy is exposed to the noisier signal of larger instances.

Waubert et al. [24] extended this idea to curriculum *within* a fixed problem size for static JSP by ranking instances according to a novel difficulty metric and demonstrating 3.2% shorter average makespans compared to randomly ordered training. This work establishes that even with a fixed scale, the order in which training instances are presented can have a material impact on final performance.

Lu et al. [19] proposed Enhanced Adaptive Staircase Curriculum Learning (EASCL) for static FJSP, using a GNN with attention and an adaptive staircase curriculum over instance scales from 10×5 to 40×10. A distinctive feature of EASCL is that it dynamically revisits the worst-performing scale with a probability proportional to the performance gap, preventing catastrophic

forgetting of small-scale competence as the curriculum advances. The method achieves strong generalisation on the Brandimarte and Hurink benchmarks.
Progressive CL with a graph attention network [3] combines phase-based scale progression with a GAT in a PPO framework for static FJSP, achieving top-two performance on six standard benchmarks (Barnes, Brandimarte, Dauzère-Pérès, Hurink edata/rdata/vdata).
In all of these works the environment is fully **static**, and the curriculum dimension is either **instance size** or instance difficulty: all jobs are known at time zero and no disruption occurs during the episode. The mechanisms that make scale-based curriculum effective in this setting (denser reward signals on small instances, monotonic difficulty across the curriculum, and prevention of catastrophic forgetting of the easier regime) are general curriculum-learning principles, and they are exactly the mechanisms that motivate our disruption-intensity curriculum in Chapter 13.

4.4 Research Gap

Combining the two directions above, a clear gap exists in the literature:

	Static environment	**Dynamic environment**
Without curriculum learning	Many works	Several works
With curriculum learning	4 works	**None**

No existing work applies curriculum learning to FJSP in a dynamic environment with machine breakdowns, job arrivals, and cancellations. Furthermore, all existing curriculum-learning works for scheduling vary **problem scale** or instance difficulty as the curriculum dimension. Existing curriculum-learning work in scheduling has focused on static JSP/FJSP settings and has varied instance scale or instance difficulty [3, 13, 19, 24], whereas dynamic-FJSP DRL work has generally trained directly on the target dynamic regime [8, 9, 18]. We therefore identify no prior work that uses a curriculum over *disruption intensity* for dynamic FJSP with arrivals, breakdowns, and cancellations. This thesis proposes a curriculum strategy along the **disruption-intensity axis**, the direct analogue of scale-based curriculum for dynamic environments. Prior scheduling curricula vary instance size or instance difficulty. In contrast, this work keeps the instance size fixed and varies the disruption intensity. Chapter 13 develops this analogy in detail and examines whether the mechanisms that support static scheduling curricula, reward density, catastrophic-forgetting prevention, and monotonic difficulty, also transfer to the disruption-intensity formulation.

Chapter 5

Problem Definition

5.1 Formal Description

Definition 1 (FJSSP). *Given a set of jobs $\mathcal{J}$ and machines $\mathcal{M}$, each job $j \in \mathcal{J}$ consists of an ordered sequence of n_j operations $\{o_{j,1}, \ldots, o_{j,n_j}\}$ with precedence constraints ($o_{j,k+1}$ cannot start before $o_{j,k}$ completes). Each operation $o_{j,k}$ has an eligible set $E_{j,k} \subseteq \mathcal{M}$ and processing times $p_{j,k,m} \in \mathbb{R}_+$ for $m \in E_{j,k}$. The objective is to assign each operation to exactly one eligible machine and determine non-preemptive start times satisfying machine exclusivity so as to minimise*

$$C_{\max} = \max_{j \in \mathcal{J}} C_j,$$

where C_j is the completion time of job j's last operation.

The above formulation generalises the classical job-shop problem by allowing each operation to be processed on any machine in an eligible subset, an extension first studied formally by Brucker and Schlie [7] under the name *job-shop scheduling with multi-purpose machines*, which is the direct predecessor of the FJSP as it is now used in the scheduling literature.

Three aspects of this definition shape the structure of the learning problem that follows. First, the eligibility set $E_{j,k}$ varies per operation rather than per job: a single job can have a tightly constrained first operation that only one or two machines can perform, followed by an operation that is eligible on every machine in $\mathcal{M}$. Routing decisions therefore have heterogeneous degrees of freedom along the operation chain and the value of a routing rule varies with the operation's position in the job. Second, processing times are not only operation- and machine-dependent but *differ across the eligible machines for the same operation*, which is precisely what makes routing non-trivial: choosing a faster machine may save processing time but congest a popular machine; choosing a slower machine may balance load at the cost of a longer individual operation. Third, the objective $C_{\max}$ is determined by the completion time of the very last job to finish, which is a global criterion: improving the schedule of the median job has no effect on the objective if the critical path is unaffected. This last property has a direct impact on the temporal structure of the reinforcement-learning problem: every decision can in principle influence the global outcome, so the credit-assignment horizon is the full episode.

The problem is NP-hard already in its non-flexible form [11]; introducing routing flexibility further enlarges the decision space. A comprehensive treatment of scheduling theory, including a discussion of the polynomial-time relaxations and the special cases that admit exact solutions, is given by Pinedo [23].

5.2 Dynamic Extensions

Beyond the static FJSSP, the simulation incorporates four categories of stochastic disruption that together model the principal sources of non-stationarity in a real manufacturing environment. Each category is parameterised independently, so the simulator can produce any mixture of disruption regimes, from fully static (no disruptions) to fully saturated, by varying the corresponding rate parameter. The four categories, summarised in Figure 9.1, are the following.

- **Machine breakdowns.** Any operational machine may fail at any time. When a breakdown is sampled, the job currently held on the spindle is interrupted, returned to the head of the re-queue list with priority over jobs that were merely waiting, and the machine is taken offline. Repair takes a random non-deterministic time, at the end of which the machine is restored and all jobs that had been queued on it are re-routed using the current routing rule. The agent must therefore reason not only about the cost of the immediate disruption but also about the long-tail re-routing decision that becomes available at the moment of repair.

- **Job cancellations.** Each initial job is independently eligible for cancellation with a fixed probability and at a random time within the first part of the horizon. Cancelled jobs are purged from all queues; if a cancellation occurs while the job is being processed, the spindle is freed and the machine returns to dispatch. Cancellations release shop-floor capacity, but the timing of the release is unpredictable from the policy's point of view.

- **Dynamic arrivals.** New jobs enter the system according to a Poisson process during the active portion of the episode. An arriving job is fully specified (operation sequence, eligibility sets, processing times) at the moment of arrival and is immediately routed to a machine using the current routing rule. The arrival process injects fresh work into the shop at moments when the policy has already committed to a partial schedule, and the policy must balance dispatching the existing backlog against accommodating the incoming work.

- **Operation completions (steady events).** The normal, non-disruptive event of a machine finishing an operation and becoming available for the next job. Although not a disruption in the literal sense, it constitutes a decision point of equal importance: every steady event triggers a fresh policy query and is responsible for the bulk of the dispatching activity during an episode.

The agent does not observe the schedule of future events; the only information it has about disruptions is the type of the current event and a global signal of how much disruption activity is still expected (the *arrival pressure* feature described in Chapter 6). This is what makes the dynamic FJSSP a genuinely sequential decision-making problem under uncertainty rather than a sequence of independent re-optimisations.

5.3 Instance Configuration

Unless otherwise noted, the simulation uses the parameters listed in Table 5.1. The instance is deliberately compact (six machines, twenty initial jobs, three operations per job) so that the exhaustive enumeration of all 25 rule combinations remains tractable as a baseline, while still exhibiting all the qualitative phenomena of a larger shop: bottleneck formation, queue build-up, breakdown propagation, and dynamic re-routing. Chapter 13 reports a separate scaling study on an 8-machine, up-to-40-job configuration, which confirms that the qualitative findings transfer to larger instances.

Table 5.1: Environment parameters used throughout all experiments.

Parameter	**Symbol**	**Value**
Machines	$\lvert\mathcal{M}\rvert$	6
Initial jobs	$\lvert\mathcal{J}_0\rvert$	20
Operations per job	n_j	3 (fixed)
Eligible machines / op	$\lvert E_{j,k}\rvert$	2–3 (random)
Processing times	$p_{j,k,m}$	$\mathcal{U}$[2.0, 15.0]
Episode horizon	$T_{\max}$	dynamic (see Chapter 8)
Breakdown rate	λ_b	0.008 per machine
Repair time	T_{repair}	$\mathcal{U}$[5, 20]
Cancellation prob.	p_c	15% per initial job
Dynamic arrival rate	λ	0.4 (Poisson, max 15)

A few of these values are worth motivating. The eligibility set size is sampled uniformly in $\{2, 3\}$ so that each operation has genuine routing flexibility (two or three machines compete for it) without trivialising the problem (any operation can be performed on any machine). Processing times are drawn from $\mathcal{U}[2.0, 15.0]$, a range broad enough that the choice between fast and slow eligible machines has significant consequences but narrow enough to avoid degenerate instances in which a single operation dominates the makespan. The breakdown rate $\lambda_b = 0.008$ produces, in expectation, one failure per machine every $1/\lambda_b = 125$ simulated time units, so a six-machine episode of horizon $T_{\max} \approx 150$ sees on the order of seven breakdown events: enough to make breakdown handling a regular concern but not so many that the shop is constantly under repair. The cancellation probability $p_c = 0.15$ produces, in expectation, three cancellations per twenty-job episode, again enough to require a learned response. Finally, the dynamic arrival rate $\lambda_a = 0.4$ and the cap of 15 total arrivals together inject substantial fresh work into the shop, on average $|\mathcal{J}_0| + 15 = 35$ jobs are processed per episode (75% more than the initial pool), without letting the episode become unboundedly long.

Chapter 6

State, Action, and System Pipeline

6.1 System Pipeline

Figure 6.1 illustrates the complete data and control flow of the system, from episode initialisation to metric logging. The agent is queried at every decision point produced by the event-driven simulation loop, not at fixed time intervals. This event-driven structure has two important consequences. First, the policy never wastes computation on timesteps at which no scheduling decision is required: an agent step is only taken when a machine has just become idle, a new job has arrived, or a disruption has changed the routing state. Second, the simulated time between successive policy queries is variable, two adjacent decisions may be separated by a fraction of a time unit (when several events fire close together) or by several time units (when the shop is fully busy and no event fires until the next operation completes). The Markov decision process induced by this loop therefore has a horizon that is governed by the number of events rather than by simulated time, and the policy must implicitly reason about variable inter-decision intervals through features that summarise the elapsed time and the current machine state.

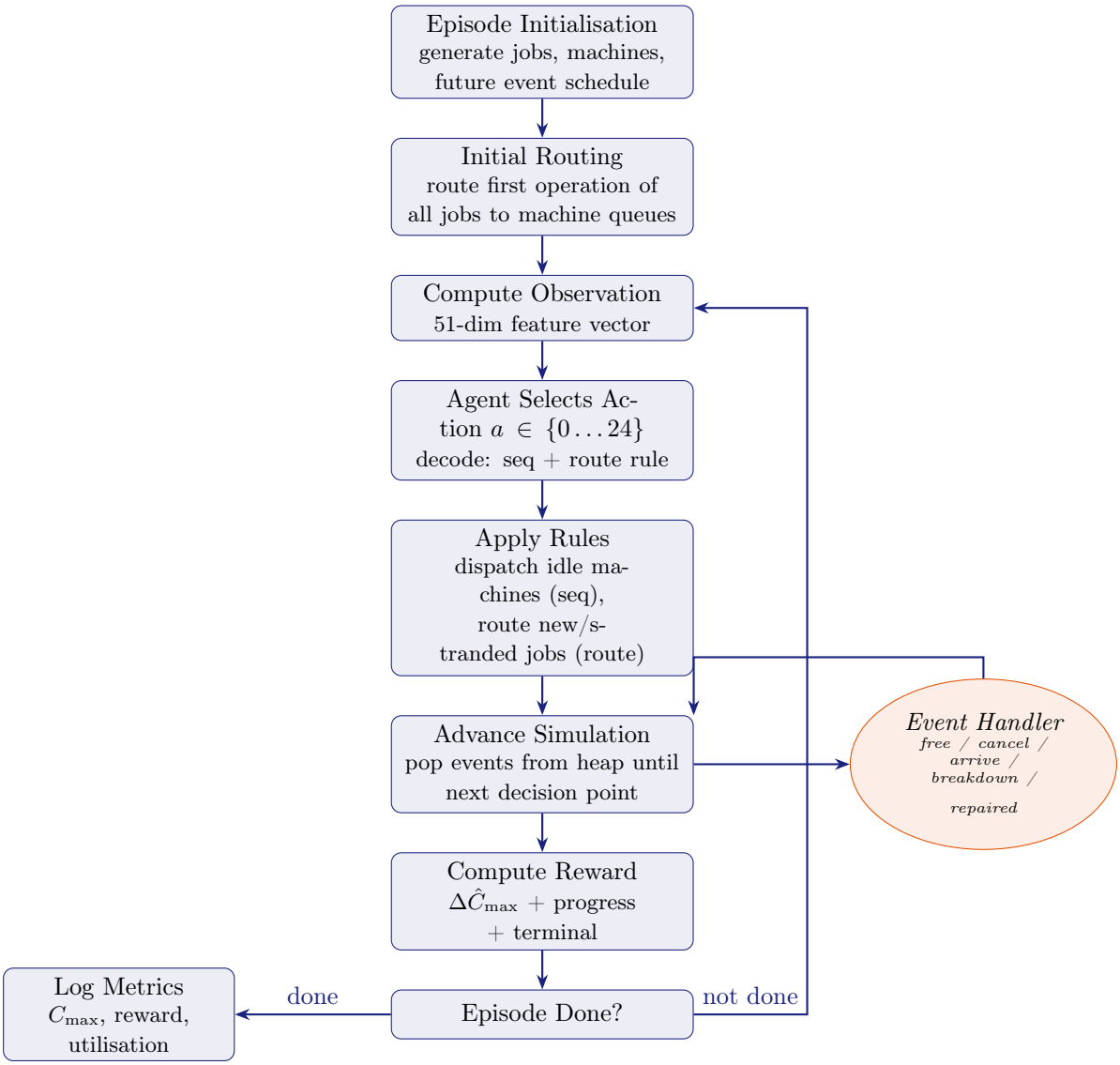
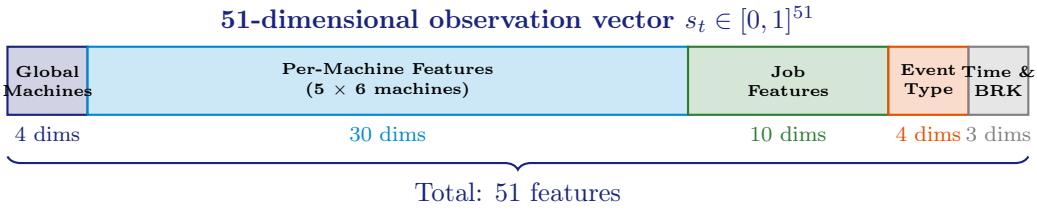


Figure 6.1: Complete system pipeline from episode initialisation to metric logging. The agent is queried at every decision point produced by the event loop.

6.2 Observation Vector (51 Dimensions)

The observation is a normalised, clipped-to-$[0, 1]$ feature vector composed of five semantic blocks (Figure 6.2). The choice of features is guided by a single design principle: every feature must be *either* directly informative for at least one of the ten rule choices the policy makes, *or* a global signal that the policy needs in order to distinguish operating regimes. Features that are correlated with neither the rule-level decisions nor the regime distinction would only inflate the input dimension and increase the sample complexity. Each feature is normalised, by the number of machines, by the reference makespan τ_{ref} (Chapter 8), or by the maximum arrival budget, and clipped to $[0, 1]$ so that input magnitudes do not depend on the instance size or its random realisation.



Block	Features
Global Machines (4)	Mean utilisation $\bar{u}$; utilisation std $2\sigma_u$; bottleneck workload fraction crit_load; fraction of busy machines $n_{\text{busy}}/\lvert\mathcal{M}\rvert$
Per-Machine (30)	For each of 6 machines: normalised queue length $\lvert Q_m\rvert$; normalised workload W_m; idle & operational (binary); broken (binary); normalised time-until-free $(b_m - t)/\tau_{\text{ref}}$
Job Features (10)	Remaining proc. time mean $\bar{r}$ and $2\sigma_r$; WIP relative load; active job count; dynamic fraction; job age mean/std; fraction on op. 1 and op. 3; *arrival pressure* ϕ
Event Type (4)	One-hot: STEADY, CANCELLATION, ARRIVAL, BREAKDOWN
Time & Breakdown (3)	Elapsed time (normalised); fraction of machines broken; mean remaining repair time

Figure 6.2: Structure of the 51-dimensional observation vector. Proportional widths in the bar chart reflect the number of dimensions in each block.

What each block contributes

The five blocks address complementary aspects of the scheduling state. Each is motivated in turn.

Global Machines (4 features). These features summarise the macroscopic load condition of the shop floor in a way that does not depend on machine identity. The *mean utilisation* $\bar{u}$ and its dispersion $2\sigma_u$ together tell the policy whether the shop is currently under-utilised (room to push more work into the queues) or already saturated; the spread σ_u in particular signals load imbalance, which is the principal symptom that the BalUtil or MinQueue routing rules are designed to correct. The *bottleneck workload fraction* crit_load identifies how much of the total remaining work is concentrated on the most loaded machine: a value close to one indicates a hard bottleneck, in which case MWKR sequencing (which prioritises jobs that still need many operations, many of which may visit the bottleneck) becomes preferable to SPT. The *fraction of busy machines* $n_{\text{busy}}/\lvert\mathcal{M}\rvert$ provides a zero-cost regime indicator that distinguishes "most machines are working, dispatching choices matter" from "most machines are idle, the binding constraint is supply of work".

Per-Machine block (30 features). This block dominates the observation vector and gives the policy the fine-grained, per-machine information that any non-trivial routing decision requires. For each of the six machines, five features are provided:

- The normalised *queue length* $|Q_m|$ is the immediate signal for MinQueue routing: a job sent to the shortest queue will have the smallest expected wait time.
- The normalised *workload* W_m (sum of queued processing times) is the corresponding signal for MinWorkload routing, which is finer-grained than MinQueue because it accounts for *how long* the queued jobs will take, not just how many there are.
- The binary *idle and operational* flag tells the policy whether a machine is currently available to dispatch. A sequencing rule cannot fire on a busy machine, so this feature determines which subset of the per-machine vector is actionable right now.
- The binary *broken* flag isolates machines that are temporarily off-line. Routing a job to a broken machine is possible (the job waits in the queue until repair) but generally undesirable; the policy must learn the appropriate avoidance behaviour from this signal.
- The normalised *time-until-free* $(b_m - t)/\tau_{\text{ref}}$ tells the policy when a currently busy machine will next become available. This is the only feature that gives the policy forward-looking information about machine availability, and it is essential for the EarliestFree routing rule: routing a job to the machine that will be free soonest is only possible if the policy knows the relative free times.

The Per-Machine block is intentionally permutation-sensitive: machine m always occupies the same position in the observation vector across the entire training and evaluation, so the network can learn machine-specific biases. This is a strength when the number of machines is fixed (the policy can recognise that, e.g., machine 3 is historically a bottleneck on this instance distribution) and a weakness when the number of machines varies, a limitation that is revisited in Section 13.8.

Job Features (10 features). These features summarise the workload from the jobs' point of view. The *mean and 2σ of remaining processing time* characterise the size and variance of the work still to do, which is the natural signal for SPT-versus-LPT sequencing: when the remaining-time distribution has a heavy left tail, SPT can clear many short jobs cheaply; when the distribution is concentrated around a high mean, LPT or MWKR are more appropriate because there are no low-cost improvements. The *WIP relative load* and *active job count* together measure how much work is currently in progress relative to the shop's capacity. The *dynamic fraction*, the share of currently active jobs that arrived dynamically rather than at episode start, is a regime indicator that complements the event-type one-hot: a high value means that the agent's recent decisions have already had to absorb several arrivals, and a higher arrival density is therefore likely in the near future. The *job age mean/std* encodes how long the active jobs have already been in the shop, an indirect tardiness signal. The two features *fraction on operation 1 and on operation 3* encode the distribution of jobs along the operation chain: a high fraction on operation 1 means many jobs are still upstream, while a high fraction on operation 3 means the remaining work is concentrated on final operations that, once dispatched, will close out the corresponding jobs. The last entry of this block, *arrival pressure* ϕ, is discussed below in detail because it is the only feature that conveys forward-looking information about future events.

Event Type (4 features). A one-hot encoding of the type of event that triggered the current decision: Steady (a routine operation completion), Cancellation (a job withdrawal), Arrival (a new job entering the shop), or Breakdown (a machine failure). This block is small (4 dimensions) but particularly informative because it tells the policy *why* it has been queried and therefore which decision is in fact at stake. The empirical analysis of the trained policy in Section 11 confirms that the agent uses this signal to switch between sharply different rules: SPT and SPT-Rt during arrivals to clear capacity for incoming jobs, FIFO and MinQueue

during breakdowns to spread the load freed by the broken machine, and the steady-state defaults otherwise. Without the event-type one-hot, the policy would have to infer the event type from collateral changes in the rest of the state, which is feasible in principle but introduces an avoidable estimation error.

Time and Breakdown (3 features). The *normalised elapsed time* t/τ_{ref} gives the policy a sense of where in the episode it currently is, which is important because some events (cancellations, arrivals) only occur during a specific window of the episode and the optimal policy late in the horizon may differ from the optimal policy near the start. The *fraction of machines broken* and the *mean remaining repair time* together describe the current breakdown state: how many machines are unavailable, and how much longer they are expected to remain so. These two features let the policy reason about the duration of the current disruption rather than only about its presence, which is essential because the appropriate response to a brief outage (temporarily route work elsewhere, plan to return) is different from the appropriate response to a long outage (treat the broken machine as permanently unavailable for the remainder of the episode).

Arrival pressure feature ϕ. The final job feature, *arrival pressure* ϕ, encodes the expected intensity of future dynamic disruptions. In this work it is implemented as the fraction of the maximum arrival budget still unused:

$$\phi = \frac{N_{\text{arr}}^{\max} - n_{\text{arrived}}}{N_{\text{arr}}^{\max}},$$

which starts at 1 at episode reset and decays monotonically to 0 as dynamic jobs arrive. A high value of ϕ signals that many arrivals are still pending, prompting the agent to be conservative in saturating machine queues; a low value indicates that the arrival phase is nearly over and the agent can optimise aggressively for the existing workload.
While the implementation here assumes knowledge of $N_{\text{arr}}^{\max}$, this is a simplification made for the simulation setting. In a real manufacturing deployment, an equivalent signal is readily available from multiple sources: the remaining capacity in the production order book, time-of-day patterns (new orders typically stop arriving near shift end), or a rolling estimate of the empirical arrival rate derived from historical MES data. Preliminary experiments suggest that removing ϕ from the observation degrades mean $C_{\max}$ by approximately 2–3 units (seed 42, 5M steps), indicating its contribution to the learned policy.

6.3 Action Space

The action space is the Cartesian product of five sequencing rules and five routing rules, yielding $5 \times 5 = 25$ joint actions. Two encodings of this joint space are considered, a flat single-head *Discrete(25)* encoding and a factorised two-head *MultiDiscrete([5,5])* encoding, and the trade-off between them is studied in detail in Chapter 7. Both encodings share the same five sequencing rules and the same five routing rules, listed below with the rationale for their inclusion.

Sequencing rules (which queued job to dispatch next)

The sequencing rule fires on an idle machine and selects the next job from its queue. The five rules are deliberately chosen to span the classical taxonomy of dispatching heuristics so that the agent can fall back to a sensible decision in any regime.

- **SPT** (Shortest Processing Time). Dispatches the queued job whose next operation has the smallest processing time on this machine. SPT minimises mean flow-time and is the canonical

choice when the goal is to clear short jobs quickly to free machine capacity for incoming work; it is therefore the natural sequencing companion to arrival-heavy regimes.

- **LPT** (Longest Processing Time). The opposite of SPT. Dispatches the longest queued job first. LPT is preferable when long jobs are on the critical path and starving them until the end of the episode would dominate the makespan.
- **MWKR** (Most Work Remaining). Dispatches the queued job whose total remaining processing time (across all its remaining operations) is largest. MWKR is the strongest single rule on this problem instance in the absence of disruptions, because giving long-tail jobs an early start usually reduces the critical-path makespan.
- **MOR** (Most Operations Remaining). Dispatches the queued job with the largest number of operations still to do. MOR is a structural complement to MWKR: it can identify critical jobs based on the number of remaining steps even when individual operation times are short.
- **FIFO** (First In, First Out). Dispatches the queued job that has been in the queue longest. FIFO is fair, has no starvation, and serves as a robust baseline that performs reasonably well across many regimes; it is often the sequencing rule of choice after a breakdown, because the agent does not need fine-grained information about job properties, it only needs to clear the backlog cleanly.

Routing rules (which eligible machine to send a job to)

The routing rule fires when a new job arrives, when a job's previous operation completes (and the next operation must be routed), or when machines are restored after a breakdown (in which case all jobs that had been queued on the broken machine must be re-routed). The five rules span the orthogonal axes of *load* (route to least busy), *speed* (route to fastest), *symmetry* (route to balance utilisation), and *availability* (route to soonest free).

- **MinWL+P** (Minimum Workload plus Processing time). Routes the job to the eligible machine that minimises the sum of the current queued workload and the new operation's processing time on that machine. This rule jointly accounts for current load and instance-specific routing speed, and is empirically the best static routing rule on the baseline instance distribution.
- **MinQueue** (Minimum Queue length). Routes to the eligible machine with the fewest queued jobs. Cruder than MinWL+P (it ignores the actual length of the queued operations), but more robust when queue lengths are short and processing times are noisy.
- **SPT-Rt** (Shortest Processing Time, Routing variant). Routes the job to the eligible machine with the smallest processing time for this specific operation, ignoring the current queue. This is the classical "go-to-the-fastest-machine" rule and is particularly valuable for newly arrived jobs that need to begin processing quickly.
- **BalUtil** (Balance Utilisation). Routes to the eligible machine with the lowest cumulative utilisation so far in the episode. Designed to keep load balanced across machines over the long horizon. Empirically the weakest routing rule on this problem (the trained policy nearly never selects it), because cumulative utilisation is a lagging indicator that does not respond to short-horizon congestion.
- **EarliestFree**. Routes to the eligible machine that will become free soonest. Differs from MinWL+P in that it accounts for the remaining processing time of the job currently on the

spindle rather than only the queued workload. Particularly useful when the eligible set is small and one machine is about to finish a long job.

Each rule is by construction a feasible scheduling primitive: it respects precedence, eligibility, and machine-exclusivity constraints. Composing two rules into a joint action does not break feasibility because the sequencing and routing rules operate on disjoint subsets of the state, the sequencing rule selects which queued job to dispatch on a currently idle machine, while the routing rule selects which eligible machine receives a new or re-queued job.

Single-head (primary). The action space is $\mathcal{A} = \{0, 1, \ldots, 24\}$ (**Discrete(25)**), encoding all rule combinations as a single integer:

$$a = 5 \cdot s + r, \qquad s, r \in \{0, 1, 2, 3, 4\}.$$

Two-head (variant). The action space is $\mathcal{A} = \{0, \ldots, 4\}^2$ (**MultiDiscrete([5, 5])**), encoding sequencing and routing as two independent variables $[s, r]$.

Sequencing Rule (s) \ Routing Rule (r)	MIN_WL	MIN_Q	SPT_R	BAL_U	EARLIEST
SPT	$a = 0$	$a = 1$	$a = 2$	$a = 3$	$a = 4$
LPT	$a = 5$	$a = 6$	$a = 7$	$a = 8$	$a = 9$
MWKR	$a = 10$	$a = 11$	$a = 12$	$a = 13$	$a = 14$
MOR	$a = 15$	$a = 16$	$a = 17$	$a = 18$	$a = 19$
FIFO	$a = 20$	$a = 21$	$a = 22$	$a = 23$	$a = 24$

shaded: $s = r$ diagonal $s = \lfloor a/5 \rfloor, \quad r = a \bmod 5$

Figure 6.3: The 25-element joint action space. Each cell shows action index a. Shaded cells lie on the diagonal where $s = r$.

6.4 Decision Points and State Transitions

The agent is queried at *discrete decision points* produced by the event-driven simulation, not at fixed time intervals. A decision point occurs when:

- a machine-free event fires and at least one idle machine can dispatch;
- a disruption event (arrival, breakdown, cancellation) changes the routing state.

After the action is decoded, (i) the sequencing rule sorts each idle machine's queue and dispatches the top job, and (ii) the routing rule directs newly arrived or re-queued jobs to eligible machines. The simulation then advances to the next decision point.

This event-driven formulation has three properties that bear directly on the reinforcement-learning problem. First, the number of decision points per episode is a stochastic function of the realised events, typically in the range of 200–500 for the small instance, and the policy must therefore be robust to variable episode lengths rather than exploit a fixed horizon. Second, the action selected at decision point t has consequences that propagate through every event that follows: a single early routing choice can shift the bottleneck machine for the remainder of the episode. This long-horizon dependency is what justifies using an undiscounted return ($\gamma = 1$) for

the makespan objective, as discussed in Chapter 8. Third, the alternative formulation in which the agent is queried at fixed time intervals would either oversample (most ticks would carry no scheduling decision and would only inflate the trajectory length) or undersample (events that fire between two ticks would be processed by a default rule rather than by the learned policy). The event-driven formulation avoids both failure modes and ensures that every policy query corresponds to a real scheduling decision.

Chapter 7

Algorithm and Model Architecture

7.1 Proximal Policy Optimisation

The agent is trained with Proximal Policy Optimisation (PPO) [27], an on-policy actor-critic algorithm that alternates between rollout collection and multiple gradient update epochs. PPO is in essence a regularised policy-gradient method: it ascends the gradient of the expected return, but constrains each update to stay within a trust region around the current policy. This trust region is enforced not through an explicit KL divergence constraint (as in TRPO) but through a simple, computationally cheap clip on the probability ratio between the new and old policies. Specifically, the clipped surrogate objective is

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\,\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\,\hat{A}_t\Big)\Big],$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\mathrm{old}}}(a_t|s_t)$ is the importance ratio between the updated policy π_θ and the behaviour policy $\pi_{\theta_{\mathrm{old}}}$ that collected the rollout, and $\varepsilon = 0.2$ is the clip parameter. The min operation ensures that the objective is a *pessimistic lower bound*: if a positive advantage estimate $\hat{A}_t$ suggests increasing the probability of action a_t, the gradient is killed as soon as the ratio exceeds $1+\varepsilon$, preventing the policy from drifting too far from the data-collection distribution. Conversely, if a negative advantage suggests decreasing the probability, the gradient is killed as soon as the ratio falls below $1-\varepsilon$. This clipping is what gives PPO its empirical robustness under high-variance gradient signals.

Advantages $\hat{A}_t$ are estimated using Generalised Advantage Estimation [26], which exposes a bias–variance trade-off through the GAE parameter λ_{GAE}. With $\lambda_{\mathrm{GAE}} = 1$ the estimator reduces to the Monte-Carlo empirical return (unbiased but high-variance); with $\lambda_{\mathrm{GAE}} = 0$ it reduces to the one-step temporal-difference target (biased but low-variance). The setting used here, $\lambda_{\mathrm{GAE}} = 0.95$, sits close to the Monte-Carlo end, which is appropriate for an environment in which the value function is relatively easy to fit but the credit-assignment horizon is long.

The full PPO loss adds a value-function term and an entropy bonus:

$$L(\theta) = L^{\mathrm{CLIP}}(\theta) - c_1\,L^{\mathrm{VF}}(\theta) + c_2\,H[\pi_\theta(\cdot|s_t)].$$

The value-function term L^{VF} is the squared error between the value head's prediction and the GAE return target. The entropy term $H[\pi_\theta]$ is the Shannon entropy of the action distribution, which is added to the objective with a small positive coefficient ($c_2 = 0.01$) to discourage premature collapse onto a single action; the policy is encouraged to maintain a non-trivial probability over all 25 actions during early training, ensuring that the value function sees a sufficiently diverse training distribution.

Why PPO for this problem. PPO is the natural algorithmic choice for three reasons related to the FJSSP simulation structure. First, the reward signal is high-variance and episode-dependent: the makespan of a single episode is sensitive to the realised disruption pattern, so two episodes started from the same initial state and trained with the same policy can have very different returns. Algorithms without a trust-region constraint (such as REINFORCE, A3C, or A2C) can be destabilised by such variance, because a single rollout with an unusually high return can produce a catastrophic policy update. PPO's clip explicitly prevents this. Second, the action space is small (25 actions) and discrete, which makes the on-policy formulation tractable: a typical rollout sees every action many times, so the policy's importance-ratio variance stays bounded. Third, the simulation is fast, a parallel rollout of 8×4096 transitions takes seconds, so the sample-efficiency disadvantage of on-policy methods compared to off-policy methods (e.g., DQN, SAC) is not binding here. The empirical comparison with DQN under matched curriculum in Section 13.13 confirms that PPO is the better algorithmic fit for this problem: our results suggest that off-policy replay buffers are disadvantaged under curriculum training, because they retain stale experience from earlier curriculum phases. A textbook reference for the broader context of policy-gradient methods is [30].

7.2 Network Architecture

Policy and value functions share a multi-layer perceptron (MLP) feature extractor. Figure 7.1 illustrates the architecture. Three choices in this design are worth motivating.

Why a shared feature extractor? The policy head and the value head solve different problems (the policy estimates the optimal action distribution; the value head estimates the expected return), but both depend on the same underlying summary of the shop-floor state. Sharing the trunk forces the two heads to agree on a common representation, which improves sample efficiency (each forward pass produces gradient signal for both objectives) and acts as a mild regulariser (features that are useful only for one head are penalised). This is the standard choice in PPO implementations and is preserved here.

Why width 256–256–128 and depth 3? The depth and width are sized to the dimension of the observation vector. With a 51-dimensional input and a 25-dimensional output, two hidden layers of width 256 give the network enough capacity to represent non-linear interactions among features (e.g., the conjunction of "machine 3 is broken" and "arrival pressure is high") while remaining within a parameter budget that PPO can train to convergence within 7 million steps. The final layer of width 128 is a deliberately narrower bottleneck that compresses the representation before the two heads split off, encouraging the network to retain only those features that are decision-relevant. Increasing the network capacity (wider hidden layers, additional layers) was tested in preliminary experiments and produced no improvement: the bottleneck on this problem is data efficiency, not representation capacity.

Why Tanh activations? The observation is normalised to $[0, 1]$ on every dimension, so the input is already in a bounded range. Tanh preserves this boundedness through the hidden layers (each Tanh output is in $[-1, 1]$), producing a network whose internal activations have well-controlled scale at every layer. This is particularly helpful for the value function, which on this problem can take extreme values (a near-zero makespan on a no-disruption episode versus a much larger makespan on an extreme-disruption episode); Tanh activations keep the representation from saturating in either direction and were empirically more stable than the alternative ReLU choice in early experiments.

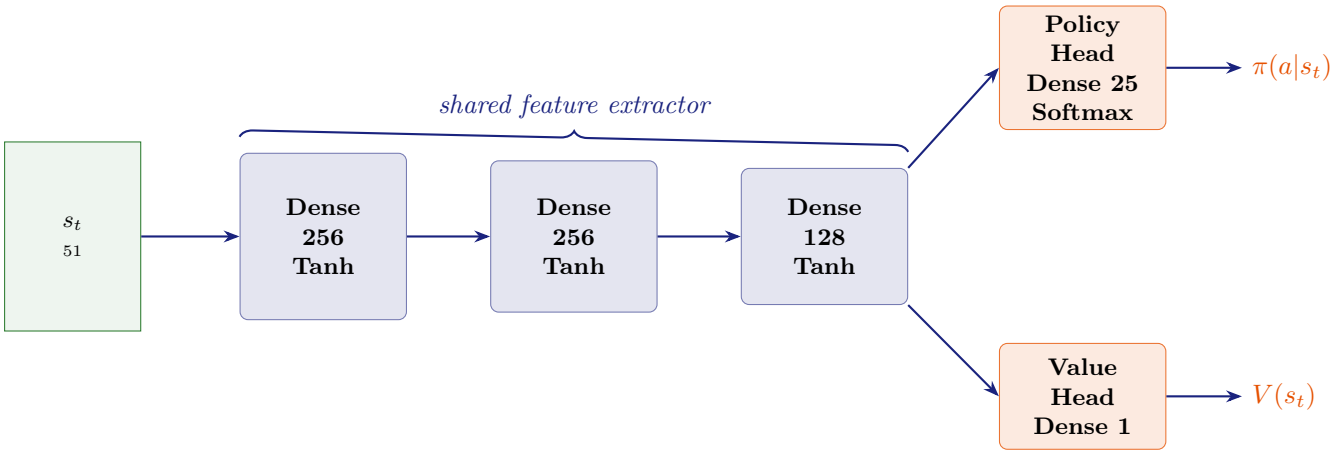


Figure 7.1: MLP architecture. Policy and value heads share the feature extractor (256–256–128, Tanh activations). Tanh keeps activations bounded, consistent with the $[0, 1]$-normalised inputs.

7.3 Single-Head Formulation (Discrete(25))

In the primary formulation the action space is a single **Discrete(25)**, encoding the Cartesian product of sequencing and routing choices as one integer:

$$a = 5 \cdot s + r, \qquad s \in \{0, \ldots, 4\},\ r \in \{0, \ldots, 4\}.$$

Stable-Baselines3 wraps this with a single *Categorical* distribution over 25 logits. The policy network outputs a 25-dimensional softmax vector:

$$\pi_\theta(a \mid s_t) = \frac{\exp(z_a)}{\sum_{a'=0}^{24} \exp(z_{a'})}, \qquad \mathbf{z} = W_{\text{policy}}\, h_3 + b,$$

where h_3 is the last shared hidden layer (dim 128).

Key property: joint learning. The network directly learns the full joint distribution over all 25 pairs. It can therefore capture *synergistic* combinations: for example, it may learn that MWKR sequencing is particularly valuable together with MIN_WORKLOAD routing, even if neither rule scores highly on its own. The marginal probabilities $P(s{=}i) = \sum_r \pi_\theta(5i{+}r)$ and $P(r{=}j) = \sum_s \pi_\theta(5s{+}j)$ are derived quantities and do not need to factorise.

Decoding at inference time.

$$s^* = \lfloor a^*/5 \rfloor, \qquad r^* = a^* \bmod 5, \qquad a^* = \arg\max_a \pi_\theta(a \mid s_t).$$

7.4 Two-Head Formulation (MultiDiscrete([5, 5]))

An alternative formulation replaces the single joint action with a **MultiDiscrete([5, 5])** space, encoding sequencing and routing as two *independent* categorical variables:

$$\mathbf{a} = [s,\ r], \qquad s, r \in \{0, 1, 2, 3, 4\}.$$

This factorisation has external support in the FJSP DRL literature: Lei et al. [15] formalise FJSP scheduling as a setting in which a single agent must produce multiple actions simultaneously, the next operation and the eligible machine to assign it to, and Liu et al. [18] go further, instantiating sequencing and routing as two separate DRL agents in their dynamic-FJSP framework. Our two-head architecture sits between these two designs: a single shared trunk but two output heads, factorising the rule-pair decision without splitting the policy into two networks.

Probability model. Stable-Baselines3 uses a *MultiCategorical* distribution: one independent Categorical per dimension. The policy network outputs *two separate* 5-dimensional softmax vectors from the final shared hidden layer:

$$\pi_\theta(\mathbf{a} \mid s_t) = \pi_s(s \mid s_t) \cdot \pi_r(r \mid s_t) = \frac{e^{u_s}}{\sum_{s'} e^{u_{s'}}} \cdot \frac{e^{v_r}}{\sum_{r'} e^{v_{r'}}},$$

where $\mathbf{u} = W_s\, h_3 + b_s \in \mathbb{R}^5$ and $\mathbf{v} = W_r\, h_3 + b_r \in \mathbb{R}^5$ are two independent linear projections from h_3. The entropy of the joint policy decomposes as $H[\pi_\theta] = H[\pi_s] + H[\pi_r]$, so the entropy bonus encourages diversity in both dimensions separately.

Architecture. The shared trunk (256–256–128, Tanh) is identical to the single-head variant. The difference lies in the output layer: instead of one Dense-25 head, there are *two* Dense-5 heads followed by independent softmax normalisation.

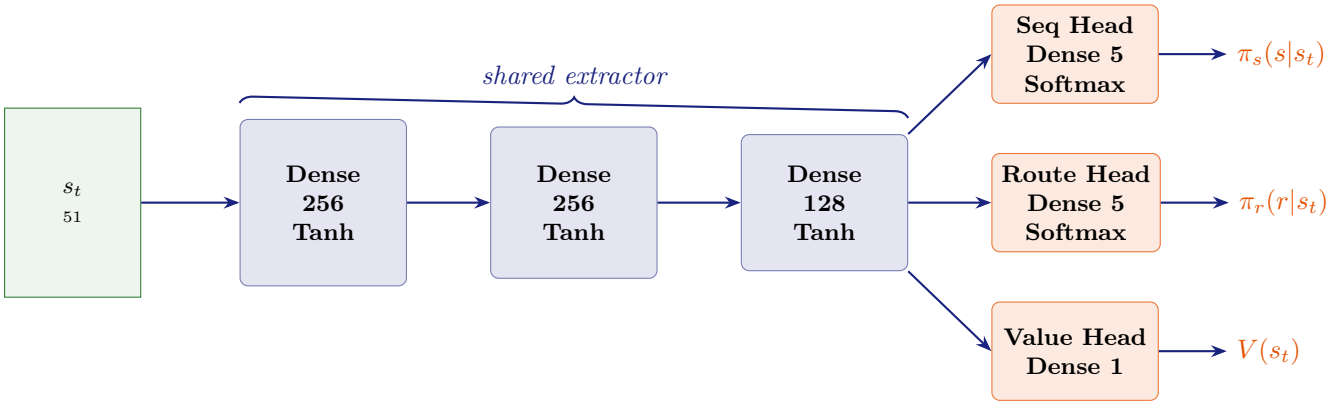


Figure 7.2: Two-head MLP architecture. The shared trunk is identical to the single-head variant; the output layer is replaced by two Dense-5 heads with independent softmax normalisation.

Key property: factorised learning. The independence assumption means $P(s, r) = P(s) \cdot P(r)$. This prevents learning synergistic pairs (e.g. MWKR specifically with MIN_WORKLOAD), but brings two practical advantages:

1. **Direct interpretability.** The sequencing and routing marginals are primary outputs rather than derived quantities, and can be read directly from the two heads without marginalising over 25 values.

2. **Gradient isolation.** The sequencing head receives a gradient signal that does not depend on which routing rule was simultaneously chosen, and vice versa. This can stabilise optimisation when the two decisions respond to different features of the state.

Action encoding at inference. The model returns a 2-element array $[s^*, r^*]$ where each component is the argmax of its respective head. The environment step call is `env.step([seq, route])` rather than `env.step(5*seq + route)`.

7.5 Comparison of the Two Formulations

Table 7.1: Single-head vs. two-head formulation summary.

Property	**Single head (Discrete(25))**	**Two heads (MultiDiscrete)**
Action encoding	One integer $a = 5s + r$	Array $[s, r]$
SB3 distribution	Categorical (25 logits)	MultiCategorical (2×5 logits)
Policy output layer	Dense-25 + Softmax	Dense-5 + Dense-5 (two Softmax)
Joint probability	Fully joint; no factorisation	Factorised: $P(s, r) = P(s) \cdot P(r)$
Synergistic combos	Learnable	Not representable
Marginal readout	Sum over 25 values	Direct output of each head
Entropy bonus	Applied to all 25 logits	Decomposes: $H_s + H_r$
Expressiveness	Higher (25 free parameters)	Lower (10 free parameters)
Interpretability	Requires marginalisation	Directly interpretable

A practical reading of this table is the following. The single-head formulation has strictly higher expressive power because it can represent any joint distribution over the 25 rule pairs, including those that do not factorise into a product of marginals. If, for example, the optimal policy is "use MWKR sequencing specifically when the routing is MinWL+P, but use SPT sequencing when the routing is SPT-Rt," then this joint dependence is representable by the single-head model but not by the two-head model. Conversely, the two-head formulation has half the output parameters and produces gradients that respect the natural separation between sequencing and routing: when an arrival fires and the routing decision dominates the return, only the routing head receives a large gradient and the sequencing head is updated only through the shared trunk. The two-head formulation is therefore better suited to problems in which the two sub-decisions are nearly independent given the state, while the single-head formulation pays off in problems where strong synergies exist between specific rule pairs. Empirically, the baseline results in Chapter 11 show that the two-head variant produces lower-variance evaluation makespans on this problem, suggesting that the sequencing and routing decisions are sufficiently decoupled that the gradient-isolation benefit outweighs the loss of expressiveness.

Chapter 8

Training Procedure

8.1 Dynamic Episode Horizon and Reward Normalisation

A key design choice is that both $T_{\max}$ and the reward normalisation reference are *instance-specific* rather than hardcoded constants. At the start of each episode, a reference value τ_{ref} is computed as

$$\tau_{\text{ref}} = \max\left(\frac{\sum_j p_j^{\text{total}}}{|\mathcal{M}|},\ \max_j p_j^{\text{total}},\ \hat{C}_{\max}^{(0)},\ 1\right),$$

where the three meaningful terms are the machine lower bound (total work divided by number of machines), the job lower bound (longest single job), and the initial optimistic $\hat{C}_{\max}$ estimate from the reset-time routing heuristic. $T_{\max}$ is set to τ_{ref}, and the same value normalises all reward components, ensuring that shaping and terminal signals are on the same scale regardless of instance difficulty.

The motivation for this construction is the following. Two instances drawn from the same distribution can have very different absolute makespans: one realisation may produce $C_{\max} \approx 70$ (few jobs, fast processing times, no breakdowns), while another may produce $C_{\max} \approx 170$ (many arrivals, several breakdowns, slow eligible machines on the critical path). If the reward were normalised by a single global constant, the gradient signal from easy instances would be small (numerator and denominator both small) and the gradient signal from hard instances would be large (both large), with the consequence that the PPO update would be dominated by hard instances and would under-fit the easy ones. Normalising by τ_{ref}, an instance-specific lower bound that is always close to the true optimum, equalises the reward scale across the instance distribution and removes this artefact. The three terms in the maximum are each individually valid lower bounds on $C_{\max}$: the total work divided by machine count is the standard "area" bound, the longest job is the standard "critical-path" bound, and the optimistic $\hat{C}_{\max}^{(0)}$ is a feasible-schedule estimate computed at reset time. Taking the maximum guarantees $\tau_{\text{ref}} \leq C_{\max}$ except in pathological cases, and the final $\max(\cdot, 1)$ prevents division by zero on degenerate instances.

8.2 Instance Generation, Dataset Split, and Generalisation

Each training episode corresponds to one **independently generated problem instance**: job processing times, operation routes, and all dynamic events (breakdowns, job arrivals, cancellations) are sampled fresh at every `env.reset()` call using a deterministic seed. There is no fixed dataset of job instances; instead, the seed controls the entire stochastic realisation, so different seeds produce structurally different scheduling problems of the same size.

Train / validation / test partition. Seeds are partitioned into three non-overlapping ranges to prevent any information leakage between splits:

- **Training** (seeds 42–49, 8 parallel environments): each rollout step draws from a continuously evolving instance. Because the environment resets with a new seed after each episode, the model is exposed to a large and diverse set of instances during training.
- **Validation** (seeds 100–199): `EvalCallback` evaluates 100 episodes every 25 000 steps and saves the best checkpoint based on mean episode reward. These seeds are never used for gradient updates.
- **Test** (seeds 9000–9299): all ablation and disruption-robustness experiments use 300 episodes drawn exclusively from this range, which was held out throughout all training runs.

One instance, one realisation per episode. Each episode consists of a single trajectory from an empty schedule to completion, using one randomly generated job configuration and one sampled realisation of future dynamic events. This is the standard episodic formulation for DRL-based scheduling [34]: the policy must generalise over the distribution of instances rather than memorise any particular one.

Generalisation within instance size. Because training, validation, and test seeds are disjoint, strong test performance directly implies that the policy generalises to *unseen* problem instances of the same size. The disruption ablation results of Chapter 12 confirm this: evaluated on 300 held-out test instances per scenario, PPO consistently matches or outperforms the best static rule, demonstrating that the learned adaptive strategy is not overfit to any specific job configuration.

8.3 Reproducibility and Seed Strategy

Full reproducibility is achieved through `numpy.random.default_rng`, which provides a seedable, statistically independent pseudo-random number generator that governs all stochastic aspects of every environment instance (processing times, disruption schedules, inter-arrival gaps, repair durations).

Training seeds. Eight parallel training environments are created by `DummyVecEnv`, each seeded with $42+i$ for $i \in \{0, \ldots, 7\}$. The fixed offset ensures the instances are deterministic but mutually independent (no shared state), giving the PPO rollout buffer diversity across problem instances within every update cycle. The evaluation environment used by `EvalCallback` is seeded at 142, outside the training range, so the policy is never evaluated on an instance it encountered during training. The SB3 model itself is also seeded at 42 (controls network weight initialisation and action sampling).

Test / evaluation seeds. After training, all 25 static rule combinations, the PPO agent, and the random baseline are each evaluated on 300 independent episodes. The static and random evaluations use `FJSSEnv()` without a fixed seed (hardware entropy), so results vary across re-runs but remain statistically stable at n=300. For exact reproduction of a single episode (e.g. the Gantt chart), the specific seed (`SEED = 42`) is passed explicitly to both `env.reset()` and the environment constructor, making that particular trajectory fully deterministic.

Multi-seed robustness. A key methodological concern is that a single training seed may reflect a particularly favourable weight initialisation rather than a systematic advantage of the algorithm. In Stable-Baselines3, the seed passed to `PPO(..., seed=SEED)` initialises *all* sources of randomness jointly, weight initialisation, environment sampling, and mini-batch shuffling, making two runs with the same seed produce bit-for-bit identical curves. To assess robustness,

both formulations were trained under two independent master seeds (42 and 123), producing four runs in total.

Summary.

Context	Seeds used	Purpose
Training envs (×8)	$42, 43, \ldots, 49$	Diverse but reproducible instances
Eval env (EvalCallback)	142	Out-of-training evaluation
PPO model	42 or 123	Reproducible weight init
Gantt / single-episode	42 (explicit)	Deterministic visualisation
Bulk evaluation (300 eps.)	hardware entropy	Statistically stable averages

8.4 Hyperparameters

The PPO hyperparameters are listed in Table 8.1. Most values are standard defaults from Stable-Baselines3 (clip parameter $\varepsilon = 0.2$, GAE parameter $\lambda_{\text{GAE}} = 0.95$, value coefficient $c_1 = 0.5$, entropy coefficient $c_2 = 0.01$, ten optimisation epochs per update), and were not separately tuned for this problem. The problem-specific choices, the discount factor, the rollout size, and the parallel-environment count, are motivated below.

Table 8.1: Training hyperparameters for both configurations.

Hyperparameter	Symbol	Value
Total timesteps	T	7 000 000
Parallel environments	N_{env}	8
Rollout steps / env	N	4 096
Transitions / update	$N \cdot N_{\text{env}}$	32 768
Mini-batch size	B	256
PPO epochs		10
Learning rate	α	3×10^{-4} (linear decay)
Discount factor	γ	1.0
GAE parameter	λ_{GAE}	0.95
Clip parameter	ε	0.2
Value coefficient	c_1	0.5
Entropy coefficient	c_2	0.01
Hidden layers		[256, 256, 128]

The discount factor $\gamma = 1.0$ (undiscounted returns) is critical for this problem. With episode lengths of approximately 200–500 decision steps, a discount of 0.995 compounds to $0.995^{500} \approx 0.08$ over a 500-step episode, effectively zeroing the credit signal for early decisions. Since makespan is a global criterion determined by the cumulative effect of all decisions, setting $\gamma = 1.0$ restores the full gradient signal and was empirically found to improve final performance by approximately 5–7 units of $C_{\max}$ compared to $\gamma = 0.995$. Optimisation uses Adam [14].

The combination of $N_{\text{env}} = 8$ parallel environments and $N = 4096$ rollout steps per environment produces a rollout buffer of $N \cdot N_{\text{env}} = 32\,768$ transitions per PPO update. Two design constraints shape this choice. First, the buffer must contain a sufficiently diverse mix of instances and disruption realisations to avoid over-fitting the update to any single trajectory, eight parallel environments seeded with consecutive integers provide enough diversity in practice. Second, the buffer must contain at least one or two complete episodes per environment so that the GAE return estimate at episode end is computed against a true terminal reward rather than

a bootstrap value. With the event-driven FJSSP structure producing $\sim$ 200–500 environment steps per episode, $N = 4096$ comfortably accommodates several episodes per rollout in every parallel environment. The resulting buffer of 32 768 transitions is then shuffled and processed in ten epochs of 128 mini-batches of size $B = 256$.

8.5 Reward Function

The reward combines step-wise shaping with terminal components:

$$r_t = \underbrace{\frac{\hat{C}_{\max}^{(t-1)} - \hat{C}_{\max}^{(t)}}{\tau_{\text{ref}}}}_{\text{step shaping}} + \underbrace{0.01 \cdot \mathbf{1}\left[\hat{C}_{\max} \text{ improved}\right]}_{\text{progress bonus}}, \tag{8.1}$$

$$r_T = \underbrace{-\frac{C_{\max}^*}{\tau_{\text{ref}}}}_{\text{makespan penalty}} + \underbrace{0.05 \cdot b_{\text{balance}}}_{\text{utilisation bonus}}, \tag{8.2}$$

where $b_{\text{balance}} = \max(0, 1 - \sigma_{\text{util}})$ rewards even machine utilisation at termination.

Each term serves a distinct purpose and addresses a separate optimisation failure mode. The *step shaping* term is the principal driver of learning: it converts the global makespan objective into a dense per-step signal by tracking the change in the online makespan estimate $\hat{C}_{\max}$. Any decision that reduces $\hat{C}_{\max}$, by dispatching a critical-path job, by routing a new operation to a machine that pulls in the critical completion time, or by re-routing a stranded job to a faster substitute machine, produces a positive reward; any decision that increases $\hat{C}_{\max}$ produces a negative reward. Without this term the agent would receive a single makespan signal only at episode termination, and the credit-assignment problem across hundreds of decision points would be intractable for any practical sample budget. The shaping is theoretically valid because it is a potential-based shaping in the sense of Ng–Harada–Russell: the total step shaping summed over an episode telescopes to the difference between the initial and final $\hat{C}_{\max}$, so it does not change the optimal policy, only the gradient signal seen during training.

The *progress bonus* is a small constant (0.01) added whenever $\hat{C}_{\max}$ strictly improves between two successive decisions. This rewards *any* step that makes measurable progress, with a magnitude that does not depend on the size of the improvement. Its role is to break ties between actions that have a comparable step shaping value: when several actions yield similar $\Delta\hat{C}_{\max}$, the agent receives an explicit signal in favour of the one that produces an actual improvement rather than a near-tie. Without this term, the policy can settle into a behaviour that makes many small but neutral decisions early in training, simply because the gradient signal on each individual action is weak.

The *makespan penalty* at episode termination provides the authoritative ground-truth objective: the agent is penalised by the final $C_{\max}^*$ normalised by τ_{ref}. Even though the step shaping has already delivered most of the signal, the telescoping sum gives the same total value across the episode, including the explicit terminal term restores the gradient through the policy's effect on the terminal state itself, which would otherwise be lost. This is particularly important for actions taken near the end of the episode, where step shaping has a short forward horizon.

The *utilisation bonus* adds a small terminal reward that rewards machine utilisation balance: σ_{util} is the standard deviation of per-machine utilisation at termination, and the bonus $0.05 \cdot \max(0, 1 - \sigma_{\text{util}})$ is positive when the utilisation is well balanced. This term is small in absolute magnitude but corrects a subtle failure mode: a policy that minimises $C_{\max}$ alone can leave several machines mostly idle while overloading one fast machine, which produces a fragile schedule (a single breakdown on the overloaded machine has catastrophic impact). The utilisation bonus

encourages the policy to distribute load across machines, which improves robustness without measurably degrading the makespan.
The online $\hat{C}_{\max}$ estimate is

$$\hat{C}_{\max}^{(t)} = \max_{m \in \mathcal{M}} \left(b_m + \sum_{j \in Q_m} p_{j,\,\text{next},\,m} \right),$$

where b_m is the time at which machine m's spindle clears, and the sum covers queued processing times of non-cancelled jobs. This is an optimistic estimate: it assumes that the queued jobs will execute in their currently queued order with no further routing changes, no further arrivals, and no breakdowns. The true $C_{\max}$ at termination is generally larger than the running $\hat{C}_{\max}$ because the simulator may inject further work or interrupt the schedule, but the difference between successive $\hat{C}_{\max}$ values is nonetheless a faithful local signal of whether the current decision is improving the achievable makespan.

8.6 Training Algorithm

Algorithm 1 summarises the training loop. Two choices beyond the standard PPO recipe are worth noting. The first is the use of $N_{\text{env}} = 8$ parallel environments that produce a single shared rollout buffer. Each environment runs its own episode independently, with a distinct training seed, so every update cycle processes $N \cdot N_{\text{env}} = 4096 \cdot 8 = 32\,768$ transitions drawn from eight different scheduling problems. This decorrelates the gradient signal (a single rollout buffer is unlikely to contain several near-identical episodes) and also gives a constant wall-clock improvement, since the simulation cost is amortised across the parallel workers. The second is the `EvalCallback` that fires every 25 000 steps, evaluates the current policy on 100 held-out validation episodes (seeds 100–199, never used for gradient updates), and saves a checkpoint whenever the validation reward improves. This "best checkpoint" rather than "last checkpoint" protocol protects against late-training degradation and is what is reported in every experimental table in this thesis.

Algorithm 1 PPO training for FJSS rule selection.

Require: Environment factory $\mathcal{E}$, hyperparameters Θ
Ensure: Trained policy π_θ

```
Initialise N_env parallel environments; initialise PPO agent
Attach EvalCallback every 25 000 steps (100 eval episodes)
EvalCallback saves best checkpoint whenever eval reward improves
for each update cycle do
    Collect N × N_env transitions {(s_t, a_t, r_t, s_{t+1})} via π_θ
    Compute GAE advantages Â_t and returns R̂_t
    for epoch = 1 to 10 do
        for each mini-batch B of size B do
            Compute L^CLIP, L^VF, entropy H
            Update θ via Adam
        end for
    end for
    Decay learning rate linearly toward 0
    if evaluation reward improved then save checkpoint
    end if
end for
return best checkpoint
```

Chapter 9

Event-Based Scheduling

This chapter describes the discrete-event simulation that drives the environment. The choice to use an event-driven rather than a time-stepped simulation is not merely an implementation detail: it shapes the structure of the Markov decision process that the policy sees, and it has direct consequences for how the agent learns.

A time-stepped simulation advances the clock by a fixed Δt at every step and re-evaluates the state at every tick. This is the natural formulation for problems with continuous dynamics (control, robotics) but is wasteful for scheduling: the overwhelming majority of ticks contain no scheduling decision (the clock simply advances and machines continue their current operations), and the few ticks that do contain decisions are sandwiched between long stretches of inaction. An event-driven simulation instead maintains a priority queue of *future events*: operation completions, planned breakdowns, scheduled arrivals, scheduled cancellations, and scheduled repairs; it advances directly to the time of the next event. The policy is queried only at the moments where a scheduling decision is genuinely required, and the trajectory length is proportional to the number of scheduling events rather than to the simulated time. This gives a tight, informative Markov decision process in which every step is decision-relevant.

9.1 Event Priority Queue

The simulation maintains a min-heap ordered by event time. Each event is a tuple (t_{event}, type, payload). Figure 9.1 shows the five event types and their causal relationships.

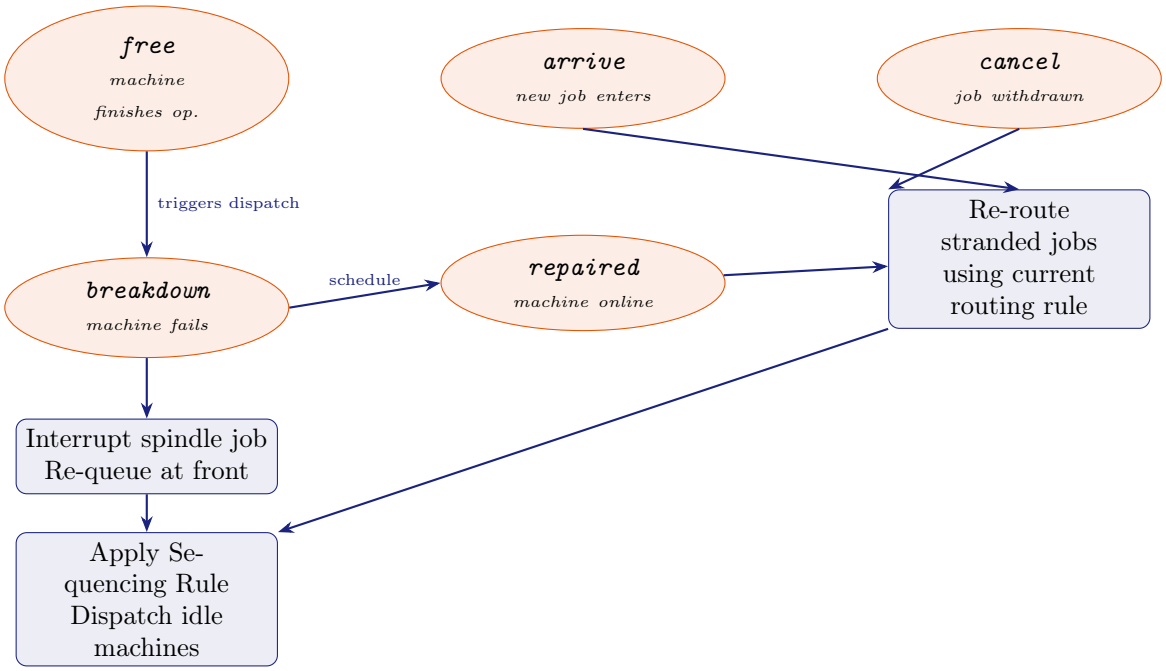


Figure 9.1: Event types and their consequences. Every event ultimately triggers a call to the sequencing and routing rule applicator.

9.2 Stochastic Event Generation

Each episode schedules its disruption events once at `reset()` time by sampling from the following distributions, all driven by the environment's seeded RNG.

Machine breakdowns. For each machine m, the time until its next failure follows an exponential distribution:

$$\Delta t_{\text{breakdown}} \sim \text{Exp}(\lambda_b), \qquad \lambda_b = 0.008.$$

The failure is only scheduled if $t_{\text{current}} + \Delta t < 0.9\, T_{\max}$, avoiding end-of-episode artefacts. After each repair the next breakdown is re-sampled, so each machine generates a renewal process of failures throughout the episode. The repair duration is drawn uniformly:

$$d_{\text{repair}} \sim \mathcal{U}[5,\, 20].$$

Dynamic job arrivals. New jobs arrive according to a Poisson process with rate $\lambda_a = 0.4$. Inter-arrival gaps are drawn as

$$\Delta t_{\text{arrive}} \sim \text{Exp}(\lambda_a),$$

and arrivals are capped at MAX_ARRIVALS $= 15$ and cut off after $0.7\, T_{\max}$ so that late-arriving jobs have time to finish.

Job cancellations. Each of the $|\mathcal{J}_0| = 20$ initial jobs is independently marked for cancellation with probability $p_c = 0.15$. Cancellation times are drawn uniformly in the first 40% of the horizon:

$$t_{\text{cancel}} \sim \mathcal{U}[1,\, 0.4\, T_{\max}].$$

Operation completions (steady events). These are not pre-scheduled; a `free` event is pushed onto the heap at time $t + p_{j,k,m}$ whenever a machine starts processing an operation.

All four event types compete in the same min-heap. The simulation always processes the chronologically earliest event, guaranteeing a consistent, globally ordered timeline.

9.3 Stale Event Detection

A subtle correctness issue arises when multiple `free` events for the same machine co-exist in the heap. Consider the following scenario: machine m starts processing operation o_1 and schedules a `free` event at time t_1. A breakdown event fires before t_1, interrupting o_1 and pushing the machine offline; the original `free` event remains on the heap but is now stale. After repair the machine resumes processing operation o_2 and schedules a fresh `free` event at t_2. The heap now contains two `free` events for m, and dispatching twice on the same machine would create an inconsistent schedule.
Each machine holds a monotonically incrementing counter `_free_event_id`. When a `free` event fires, it is silently discarded if its ID does not match the machine's current ID, preventing double-dispatch. This is logged as Fix 2 in the source code and is one of several event-ordering invariants that the simulator must maintain in order to remain a faithful generator for the RL environment.

9.4 Breakdown and Re-routing Procedure

1. Mark machine m as broken; cancel any pending `free` event.
2. If m has a spindle job: subtract unspent processing time from utilisation statistics (Fix 7); re-insert the interrupted job at the *front* of the re-queue list (Fix 4) so it regains priority after repair.
3. Collect all jobs from m's queue.
4. Schedule a `repaired` event at $t + \text{repair_time}$.
5. On `repaired`: re-route all collected jobs using the current routing rule, allowing the agent's most recent action to redirect work away from the formerly bottlenecked machine.

Chapter 10

Implementation Details

This chapter documents the engineering choices that the rest of the thesis builds on. The implementation aims at three properties: a clean separation between the simulation and the learning algorithm, a small number of well-defined data classes that carry the entire scheduling state, and full reproducibility from a single seed. Stable-Baselines3 is used as the PPO backend so that the experimental work can focus on the environment, the reward, and the curriculum rather than on reimplementing standard machine-learning infrastructure.

10.1 Module Structure

```
semester_project/
  env/
    __init__.py
    fjss_env.py           # FJSSEnv: Gymnasium environment (single-head)
    fjss_env_two_head.py  # FJSSEnv: Gymnasium environment (two-head)
  train.py                # PPO training, evaluation, and logging (single-head)
  train_two_head.py       # PPO training, evaluation, and logging (two-head)
  gantt.py                # Gantt visualisation + RecordingFJSSEnv
  requirements.txt
```

Listing 10.1: Project layout.

10.2 Key Data Structures

```
@dataclass
class Operation:
    op_id: int
    eligible_machines: Dict[int, float]    # {machine_id: proc_time}

    @property
    def min_time(self) -> float: ...       # fastest eligible machine
    @property
    def n_eligible(self) -> int: ...       # |E_{j,k}|

@dataclass
class Job:
    job_id: int
```

```
    operations: List[Operation]
    current_op: int = 0
    arrival_time: float = 0.0
    cancelled: bool = False
    completed: bool = False
    is_dynamic: bool = False

    @property
    def remaining_proc_time(self) -> float:
        # optimistic: fastest path for all remaining ops

@dataclass
class Machine:
    machine_id: int
    queue: List[Job]
    current_job: Optional[Job] = None     # FIX 3: spindle tracking
    broken: bool = False
    busy_until: float = 0.0
    busy_time: float = 0.0                # cumulative (for utilisation)
    _free_event_id: int = 0               # FIX 2: stale event detection

    @property
    def workload(self) -> float: ...      # sum of queued proc times
```

Listing 10.2: Core dataclasses (simplified).

Chapter 11

Baseline Results and Policy Analysis

11.1 Evaluation Protocol

After training, all 25 static rule combinations are evaluated on 300 independent episodes each. The PPO agent is evaluated in deterministic mode (argmax action). The metrics reported are:

- Mean and standard deviation of episode reward and final makespan $C_{\max}$.
- **Gap vs. best static**: $\Delta_{\text{static}} = (C_{\max}^{\text{PPO}} - C_{\max}^{\text{best-static}})/C_{\max}^{\text{best-static}} \times 100\%$.
- **Gap vs. random**: a sanity check that RL training succeeded.

Three aspects of this protocol are worth highlighting. First, the gap is measured against the *best* of the 25 static rule pairs identified by exhaustive sweep on the same evaluation episodes, not against a fixed reference rule. This is a strictly more demanding baseline than "vs. SPT" or "vs. FIFO": the PPO agent must beat the rule that would have been chosen by a hyperparameter sweep, not a random rule. Second, the comparison uses the *same* 300 evaluation episodes for every static rule and for PPO, so the gap is a paired comparison and is not affected by between-episode variance. Third, the PPO agent is evaluated deterministically (argmax over the policy distribution) rather than by sampling; this removes one source of evaluation noise and isolates the contribution of the learned policy from the entropy bonus that was used during training.

11.2 Quantitative Results

Table 11.1 summarises the performance of both architectures across two independent training seeds (42 and 123) at 5 M steps, compared against the best static rule pair and a random baseline. The best static combination identified by exhaustive sweep across all 25 pairs is consistently MWKR + MinWorkload+Proc with a mean $C_{\max}$ of approximately 150.

Table 11.1: Evaluation results over 300 independent episodes at 5 M training steps. Lower $C_{\max}$ is better. Baselines are averaged over 300 episodes. Gap is relative to the best static rule pair.

Method	Seed	Mean $C_{\max}$	Std	vs. best static	vs. random
Baselines (averaged across evaluation episodes)					
MWKR + MinWL+P	—	≈ 150.1	± 13.3	—	—
Random policy	—	≈ 161.5	± 16.5	+7.5%	—
Single-head `Discrete(25)`					
	42	148.23	± 13.01	−1.04%	+8.01%
	123	147.11	± 14.39	−1.94%	+8.46%
Two-head `MultiDiscrete([5,5])`					
	42	145.29	± 13.32	−3.43%	+10.43%
	123	146.35	± 12.88	−2.73%	+8.90%

Several observations follow from Table 11.1. First, both architectures beat the best static rule combination and the random baseline across *both* seeds, confirming that the improvement is not an artefact of a single favourable weight initialisation. Second, the two-head architecture outperforms the single-head consistently: the gap ranges from 1.5 to 2.4 units of $C_{\max}$ across the two seeds. Third, the two-head model produces lower variance in both seeds (± 12.88–13.32 vs. ± 13.01–14.39), consistent with the gradient-isolation hypothesis: factoring the output into independent sequencing and routing heads leads to more stable scheduling decisions.

11.3 Context-Sensitive Policy Behaviour

A central claim of this work is that the PPO agent learns a genuinely context-sensitive hyper-heuristic, one that adapts its rule selection to the type of scheduling event rather than defaulting to a single fixed combination. Table 11.2 reports the dominant rule per event type for the two-head model, averaged over 300 evaluation episodes.

Table 11.2: Dominant rule per event type. Two-head PPO (seed 42, 5 M steps), averaged over 300 evaluation episodes. Values in parentheses are marginal probabilities.

Event	Steps	Preferred sequencing	Preferred routing
Steady	30 374	MWKR (0.312)	MinWL+P (0.369)
Arrival	4 500	SPT (0.272)	SPT-Rt (0.340)
Cancellation	513	LPT (0.228)	MinQueue (0.315)
Breakdown	1 664	FIFO (0.241)	MinQueue (0.284)

Under steady-state conditions the agent defaults to MWKR + MinWL+P, consistent with the best static baseline. At arrival events it switches to SPT sequencing, clearing fast jobs to free machines for the incoming work, and routes new jobs to the fastest eligible machine (SPT-Rt), minimising their time-to-start. During breakdowns the agent shifts to FIFO sequencing and MinQueue routing, distributing the load from the failed machine across those with the shortest queues, avoiding creating new bottlenecks. At cancellations it favours MinQueue routing (0.315) to redistribute the freed capacity as evenly as possible. None of these patterns can be replicated by any single fixed rule pair; this is precisely the capability that motivates the RL approach.
It is also worth noting what the table does *not* show. The marginal probability of the preferred rule never exceeds approximately 0.37 in any event class, which means the policy does not collapse

onto a deterministic rule even after 5 M training steps. This residual diversity is partly the effect of the entropy bonus and partly a genuine policy behaviour: the agent's choice depends not only on the event type but also on the current shop-floor state. The same arrival event triggers SPT-Rt when the eligible machines have very different processing times for the new operation, but MinQueue or EarliestFree when the eligible machines are roughly equivalent. The event-type classification in Table 11.2 captures the dominant behaviour but smooths over this finer-grained context dependence.

11.4 Policy Interpretability

The marginal action probabilities reveal which rules the learned policy prefers:

$$P(s = i) = \sum_{r=0}^{4} \pi_\theta(5i + r \mid \bar{s}), \qquad P(r = j) = \sum_{s=0}^{4} \pi_\theta(5s + j \mid \bar{s}).$$

For the single-head model (seed 42, 5 M steps), the globally preferred rules are MWKR for sequencing ($P = 0.280$) and MinWL+P for routing ($P = 0.320$), consistent with the best static baseline. The policy additionally avoids BalUtil routing across all event types ($P < 0.06$ globally), having learned that historical utilisation balance is a poor proxy for short-horizon makespan minimisation.

11.5 Schedule Visualisation

Figure 11.1 shows the three-panel Gantt chart produced by `gantt.py` for a single episode (seed 42), comparing the PPO agent, the best static rule pair identified by sweeping all 25 combinations, and a random baseline.

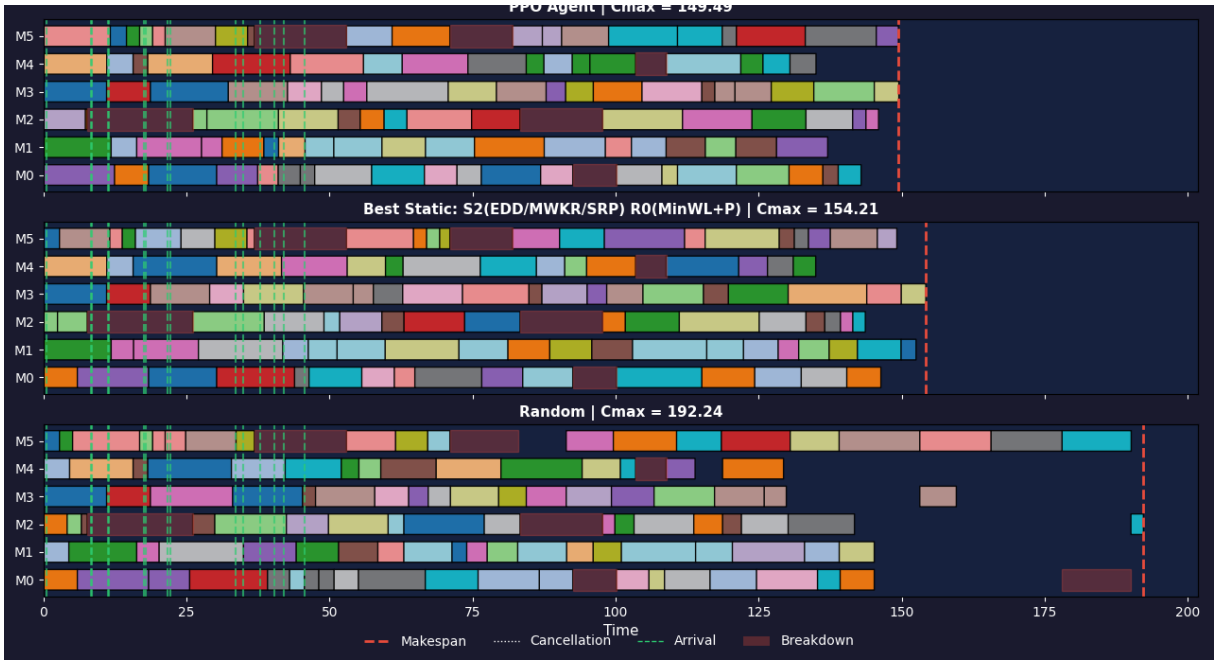


Figure 11.1: Three-panel Gantt chart (seed 42). **Top:** PPO agent (deterministic), $C_{\max} = 149.49$. **Middle:** best static rule pair (MWKR + MinWorkload+Proc), $C_{\max} = 154.21$. **Bottom:** random baseline, $C_{\max} = 192.24$. Each coloured bar is one operation; colours are consistent across panels so the same job can be tracked across policies. The red dashed vertical line marks the final makespan $C_{\max}$ for each policy. Vertical dotted lines indicate cancellation, arrival, and breakdown events.

The Gantt chart confirms the quantitative results on this single instance: the PPO agent achieves a makespan of 149.49 against 154.21 for the best static pair (-3.1%) and 192.24 for the random baseline (-22.2%). Qualitatively, the PPO schedule exhibits denser machine utilisation with fewer idle gaps, particularly in the middle of the episode when dynamic arrivals and a breakdown event occur simultaneously.

Chapter 12

Disruption Robustness Ablation (Zero-Shot)

12.1 Motivation and Protocol

The results reported in Chapter 11 were obtained on the fixed training distribution (breakdown rate $\lambda_b = 0.008$, arrival rate $\lambda_a = 0.4$, cancellation probability $p_c = 0.15$). On that distribution the PPO agent outperforms the best static rule by a margin of approximately 3–4%, which is a credible improvement on a problem where the best static rule was identified by exhaustive sweep over 25 candidates.

A natural follow-up question is whether the learned policy generalises to disruption profiles that were *not* seen during training. The training distribution exposes the policy to a single operating point in the three-dimensional disruption space $(\lambda_b, \lambda_a, p_c)$. A practical deployment would face a range of operating points throughout the workday: nights and weekends might see low arrival rates, peak production periods might see high arrival rates, scheduled maintenance windows might see elevated breakdown rates, and so on. If the agent must be retrained at every change of operating point, the deployment cost would be prohibitive, the entire promise of an adaptive hyper-heuristic is that one trained model handles many operating regimes.

To answer this question, the best-checkpoint models (single-head and two-head, seed 42) are evaluated **without any retraining** on six modified disruption configurations. Because all scenarios retain the same number of machines ($|\mathcal{M}| = 6$), the observation dimension is unchanged (51 dims) and the network receives valid inputs in every case, making this a genuine zero-shot generalisation test.

For each scenario, all 25 static rule pairs are also evaluated (100 episodes each) so that the PPO gap is always measured against the best available heuristic *on that specific scenario*, not the baseline heuristic. This is important because the best static rule changes slightly with the disruption profile, and comparing PPO against a fixed reference would unfairly disadvantage it on scenarios where that reference is not the best baseline.

12.2 Scenarios

Table 12.1: Disruption configurations used in the zero-shot ablation. The baseline row matches the training distribution exactly.

Scenario	λ_b	λ_a	p_c
Baseline (training config)	0.008	0.4	0.15
No disruption	0.000	0.0	0.00
Only breakdowns	0.016	0.0	0.00
Only arrivals	0.000	0.8	0.00
Only cancellations	0.000	0.0	0.30
Extreme disruption (all doubled)	0.016	0.8	0.30

12.3 Results

Table 12.2: Zero-shot disruption robustness. For each scenario the best static rule pair is found by exhaustive sweep; the gap is $(C_{\max}^{\text{PPO}} - C_{\max}^{\text{best-static}})/C_{\max}^{\text{best-static}} \times 100\%$. Negative gap means PPO outperforms the best static rule.

Scenario	**Single-head**		**Two-head**	
	$C_{\max}^{\text{PPO}}$	Gap	$C_{\max}^{\text{PPO}}$	Gap
Baseline	146.12	−3.6%	145.70	−3.9%
No disruption	81.69	+6.4%	80.79	+5.2%
Only breakdowns	111.57	+4.1%	110.04	+2.7%
Only arrivals	124.35	−3.9%	126.34	−2.3%
Only cancellations	77.98	+6.1%	77.73	+5.8%
Extreme disruption	165.34	−1.4%	164.81	−1.7%

Best static rule is MWKR + MinWL+P in five of six scenarios; MWKR + MinQueue achieves the lowest $C_{\max}$ under *Only breakdowns* (per-scenario screening).

12.4 Analysis

The results reveal a clear pattern: the PPO agent outperforms the best static rule whenever **dynamic job arrivals** are present (baseline, only arrivals, extreme disruption), but loses to MWKR + MinWL+P in scenarios without arrivals (no disruption, only breakdowns, only cancellations).

This behaviour is consistent with the training distribution. The agent was trained on a fixed mix in which dynamic arrivals are always present ($\lambda_a = 0.4$); its policy has therefore specialised in managing the additional routing decisions triggered by new jobs. When arrivals are removed, the scheduling problem reduces to a quasi-static instance for which the globally best fixed rule is sufficient and the context switching learned by the PPO agent provides no additional benefit.

The result is informative not only as a negative finding but because it identifies the *specific* feature of the training distribution responsible for the failure. It is not the case that the policy is brittle in general, it generalises across breakdown rates and cancellation probabilities reasonably well, and it generalises across arrival rates (only-arrivals at $\lambda_a = 0.8$ is double the training rate,

and the policy still wins by 2.3–3.9%). What it does not generalise across is the *absence* of arrivals: a regime in which one of the three disruption types is exactly zero takes the problem outside the convex hull of the training data, and the policy defaults to a context-switching strategy that provides no benefit when no arrivals occur. This diagnosis directly motivates the curriculum proposed in Chapter 13, in which the agent is explicitly trained on episodes with zero disruptions (Phase 1) before being exposed to the full disruption regime.

The two-head architecture outperforms the single-head in four out of six scenarios, with the largest advantage on the out-of-distribution cases (no disruption +5.2% vs. +6.4%; only breakdowns +2.7% vs. +4.1%). This is consistent with the gradient-isolation hypothesis introduced in Section 7.4: the factorised sequencing and routing heads each receive a cleaner gradient signal than the joint head, which helps the policy retain the structural separation between sequencing and routing under distribution shift.

12.5 Identified Limitation and Proposed Fix

The zero-shot ablation exposes a generalisation gap: the policy is **specialised for the training disruption distribution** and does not automatically transfer to scenarios with a different disruption profile.

The root cause is that all eight parallel training environments use the same fixed disruption parameters. A straightforward remedy is **domain randomisation**: at each `reset()` call, the disruption parameters are sampled from a broad distribution rather than fixed:

$$\lambda_b \sim \mathcal{U}[0,\, 0.016], \quad \lambda_a \sim \mathcal{U}[0,\, 0.8], \quad p_c \sim \mathcal{U}[0,\, 0.30].$$

This forces the agent to learn a policy that performs well across the entire disruption space rather than at a single operating point, directly addressing the observed failure modes. Additionally, using diverse parallel environments, for example, two environments with high disruption, two with low disruption, two static, and two at the training baseline, would ensure that every PPO rollout buffer contains experience from multiple disruption regimes, improving the gradient signal for generalisation. Both modifications are pursued in Chapter 13 below.

Chapter 13

Curriculum Learning and Experimental Evaluation

This chapter presents the main contribution of the thesis. We first revisit the two domain-randomisation strategies suggested in Chapter 12, pure dynamic training and mixed dynamic/static training, to confirm empirically that neither is sufficient. We then introduce the progressive curriculum along the disruption-intensity axis, demonstrate that it wins all six disruption scenarios, and provide an extensive set of follow-up studies: scale, generalisation, 2×2 ablation, phase counts, and algorithm comparison with DQN.

13.1 Approach and Experimental Setup

13.1.1 Problem Setting Recap

We consider the FJSSP with dynamic events, job arrivals, cancellations, and machine breakdowns. The objective is to minimise the makespan $C_{\max}$. At each decision step, the agent selects:

- a sequencing rule (SPT, LPT, MWKR, MOR, FIFO);
- a routing rule (MinWL+P, MinQueue, SPT-Rt, BalUtil, EarlFree).

This results in a MultiDiscrete action space of 25 possible rule combinations.

13.1.2 Training Strategies Considered

We investigate three main strategies in this chapter:

Dynamic-Only Training (Pure RL)

The PPO agent is trained exclusively on stochastic environments with domain randomization. All environments include dynamic events such as arrivals, cancellations, and breakdowns. The goal is to learn a fully adaptive policy that can respond to varying system conditions without relying on fixed heuristics.

Mixed Training (Dynamic + Static Environments)

A mixture of environments is introduced:

- 6 dynamic environments (with disruptions);

- 2 static environments (no arrivals, breakdowns, or cancellations).

The motivation is to improve performance in stable scenarios, where static heuristics are known to perform well.

Progressive Curriculum (Section 13.5 below)

A three-phase curriculum that progressively increases disruption intensity across training. This is the main contribution of the thesis and is introduced after motivating it through the limitations of the previous two strategies.

13.1.3 Baseline Comparison

For all experiments in this chapter, the learned policy is compared against:

- all 25 static rule combinations;
- a random policy baseline.

The best static rule across all experiments is consistently

$$\text{MWKR} + \text{MinWL+P}.$$

13.2 Results: Dynamic-Only Training

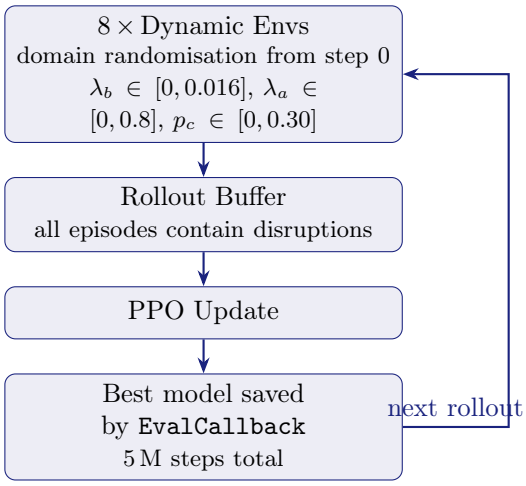


Figure 13.1: Dynamic-only training pipeline. All eight parallel environments apply full domain randomisation at every episode reset; the disruption parameters are never zero.

Table 13.1: PPO vs. static baselines (dynamic training).

Model	$C_{\max}$	Gap vs. best static
PPO (Dynamic)	$\mathbf{144.55 \pm 13.46}$	$\mathbf{-3.92\%}$
Best static	150.46	0%
Random	161.49	+7.3%

The PPO agent outperforms all static rule combinations, demonstrating the benefit of adaptive decision-making. The aggregate −3.92% improvement over the best static rule is, on its own,

a strong baseline result for an adaptive scheduler, but as the following table makes clear, the aggregate hides a structural asymmetry: the improvement is driven by scenarios containing dynamic arrivals, while scenarios without arrivals are essentially tied with, or slightly behind, the best static rule.

13.2.1 Ablation Study Across Scenarios

Table 13.2: Performance across different disruption scenarios (dynamic-only training).

Scenario	Best static	PPO $C_{\max}$	Gap
Baseline	151.56	**144.24**	−4.8%
No disruption	76.77	80.74	+5.2%
Only breakdowns	107.19	107.80	+0.6%
Only arrivals	129.37	**123.74**	−4.3%
Only cancellations	73.48	77.50	+5.5%
Extreme disruption	167.66	**164.56**	−1.9%

The results show that PPO excels in dynamic environments, particularly under job arrivals and severe disruptions. However, it underperforms static heuristics in stable scenarios. The +5.2% gap on *No disruption* and the +5.5% gap on *Only cancellations* are the largest deviations from the best static rule observed in any configuration, and they are diagnostic of a specific failure mode: the policy has learned to actively re-route work in anticipation of arrivals and to compensate for breakdowns, but those reactions are counter-productive when no such events occur. In a fully static environment the optimal strategy is simply to apply MWKR + MinWL+P consistently, there is no information in the state that justifies deviating from it. The dynamic-trained policy nonetheless deviates, because deviation is what was rewarded during training. This is the exact phenomenon that motivates domain randomisation, and the mixed training experiment in the next section is its first attempt to address it.

13.3 Results: Mixed Training (6 Dynamic + 2 Static)

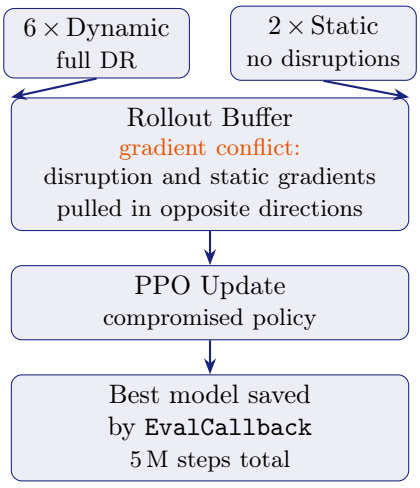


Figure 13.2: Mixed training pipeline (6 + 2). Six dynamic and two static environments contribute to the same rollout buffer each iteration. The opposing gradient signals cause a conflict that degrades performance in both dynamic and static scenarios.

Table 13.3: Mixed training results.

Scenario	Best static	PPO $C_{\max}$	Gap
Baseline	151.56	150.69	−0.6%
No disruption	76.77	77.13	+0.5%
Only breakdowns	107.19	110.05	+2.7%
Only arrivals	129.37	127.45	−1.5%
Only cancellations	73.48	73.73	+0.3%
Extreme disruption	167.66	169.49	+1.1%

Contrary to expectations, introducing static environments during training did not improve performance in stable scenarios. Instead, it *degraded* performance in both static and dynamic settings: the gap on the baseline configuration collapses from −4.8% (dynamic-only) to −0.6% (mixed), and the gap on *Only breakdowns* worsens from +0.6% to +2.7%. The expected gains on static scenarios are modest: *No disruption* improves from +5.2% to +0.5% and *Only cancellations* from +5.5% to +0.3%, but neither crosses zero. The net effect is that mixed training trades a large win on dynamic scenarios for a small win on static scenarios, and the trade is unfavourable.
The cause of this regression is a *gradient conflict* inside the shared rollout buffer. Each PPO update operates on transitions drawn from all eight parallel environments simultaneously. When six of those environments are dynamic and two are fully static, the dynamic environments produce a gradient signal that rewards context-switching behaviour (e.g., switch to SPT at arrivals, switch to FIFO at breakdowns), while the static environments produce a gradient signal that rewards holding MWKR + MinWL+P throughout the episode. The two signals are not merely independent: they point in opposite directions in the policy's parameter space. The policy update therefore receives mixed information and converges to a compromise that is sub-optimal for both regimes, it switches rules too aggressively for the static scenarios and too tentatively for the dynamic scenarios.
This is the empirical evidence that motivates the curriculum-based approach below: instead of mixing static and dynamic experience within the same rollout, the curriculum separates them temporally, letting the policy first acquire a strong static base in Phase 1 and then progressively learn to extend it with disruption-handling competence in Phases 2 and 3. Crucially, the environment swap at each phase boundary occurs at the end of a complete rollout buffer, so every PPO update sees a homogeneous batch of experience drawn from a single phase. The gradient-conflict pathology observed in mixed training is therefore avoided by construction.

13.4 Curriculum Learning in Scheduling: Prior Art and Our Rationale

Why curriculum learning works for static JSP/FJSP. The general idea of curriculum learning, presenting training examples in a meaningful order from easy to hard rather than uniformly at random, was introduced by Bengio et al. [4] and has since been formalised for reinforcement learning by Narvekar et al. [21], whose framework and survey covers the full taxonomy of task-sequencing, transfer, and generation strategies on which the present chapter builds.
Curriculum learning was introduced to scheduling by Iklassov et al. [13], who trained a DRL dispatching agent on the static JSP by sequencing training instances from small to large. They observed that an agent exposed directly to large instances receives a *sparse and noisy reward signal*: the episode is long, the action space is large, and the agent rarely stumbles upon good solutions by chance. Training on small instances first provides a dense, clear learning signal that allows the policy to learn basic scheduling behaviour before facing harder instances.

Waubert et al. [24] extended this idea to curriculum *within* a fixed problem size for static JSP, ranking instances by a novel difficulty metric. Agents trained on the best curricula achieved 3.2% shorter average makespans compared to agents trained on randomly ordered data, demonstrating that even within a fixed scale, the order of training instances matters.
Lu et al. [19] proposed the Enhanced Adaptive Staircase Curriculum Learning (EASCL) algorithm for static FJSP, using a GNN with attention and a curriculum over instance scales from 10×5 to 40×10. Critically, EASCL dynamically revisits the worst-performing scale with probability proportional to the performance gap, preventing catastrophic forgetting. The method achieves strong generalisation on the Brandimarte and Hurink benchmarks.
Progressive CL with a graph attention network [3] combines progressive curriculum phases with a GAT in a PPO framework for static FJSP, achieving top-two performance on six standard benchmarks (Barnes, Brandimarte, Dauzère-Pérès, Hurink edata/rdata/vdata).
In all these works the environment is fully **static**, and the curriculum dimension is either **instance size** or instance difficulty: all jobs are known at time zero and no disruption occurs during the episode.
Three mechanisms explain why scale-based curriculum works:

1. **Reward density.** Small instances produce shorter episodes with more frequent terminal rewards, accelerating early learning.

2. **Catastrophic forgetting prevention.** Revisiting smaller instances periodically (as in EASCL [19]) prevents the policy from losing competence on easy cases as it adapts to hard ones.

3. **Monotonic difficulty.** Each curriculum step introduces a strictly harder problem, so the policy always has a solid foundation before advancing.

Why scale-based curriculum does not apply here. Our setting fixes the instance size (6 machines, 20 initial jobs) throughout training, the same configuration at every episode. A scale curriculum would therefore have nothing to vary. The source of difficulty in our problem is not instance size but **disruption intensity**: how frequently machines break down, how many new jobs arrive, and how often jobs are cancelled. A fully disrupted episode is substantially harder than a static one: the agent must recover from unexpected events, re-route stranded jobs, and handle a dynamically changing job pool, all of which are absent from a zero-disruption episode.

Disruption intensity as the natural curriculum axis. The analogy with scale-based curriculum is direct:

Static JSP/FJSP (prior work)	**Dynamic FJSP (this work)**
Easy = small instance (10×5)	Easy = no disruptions ($\lambda_b = 0$, $\lambda_a = 0$, $p_c = 0$)
Hard = large instance (40×10)	Hard = full disruption regime
Curriculum axis = problem scale	Curriculum axis = disruption intensity

The same three mechanisms apply:

1. **Reward density.** Static episodes are shorter and provide a cleaner makespan signal, letting the policy learn basic scheduling before handling the additional complexity of disruptions.

2. **Catastrophic forgetting prevention.** Without Phase 1, the agent never internalises a strong static scheduling strategy. As shown in Table 13.10, models trained without curriculum (Models A and C) lose to static rules on zero-disruption scenarios (+5.0% and +5.2% respectively).

3. **Monotonic difficulty.** Each phase introduces strictly more disruptive conditions, so the policy always builds on a stable foundation.

This is, to our knowledge, the first curriculum learning strategy defined along the **disruption-intensity axis** for any scheduling problem.

13.5 Results: Progressive Curriculum Training

The failure of mixed training suggested that the fundamental issue was a **gradient conflict**: disruption-heavy episodes and disruption-free episodes push the policy in opposite directions when trained simultaneously. To resolve this, a three-phase **progressive curriculum** was introduced:

1. **Phase 1** (steps 0–2 M): eight static environments, zero disruptions. The policy first learns pure scheduling.
2. **Phase 2** (steps 2 M–4.5 M): domain randomisation with reduced ranges ($\lambda_b \in [0, 0.008]$, $\lambda_a \in [0, 0.4]$, $p_c \in [0, 0.15]$). The policy is gradually exposed to mild disruptions.
3. **Phase 3** (steps 4.5 M–7 M): full domain randomisation ($\lambda_b \in [0, 0.016]$, $\lambda_a \in [0, 0.8]$, $p_c \in [0, 0.30]$). The policy must handle the complete disruption regime.

The environment is replaced at phase boundaries using a `CurriculumCallback` that fires at the end of each rollout buffer, ensuring every PPO update operates on a homogeneous batch of experience.

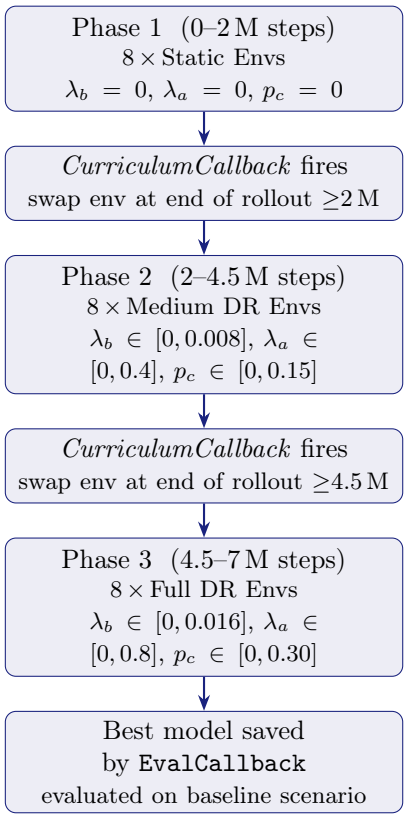


Figure 13.3: Progressive curriculum training pipeline. The `CurriculumCallback` replaces the training environment at the end of a complete rollout buffer (never mid-rollout), so each PPO update sees a homogeneous batch. Disruption difficulty increases monotonically across the three phases.

Table 13.4: Zero-shot disruption robustness, curriculum-trained two-head PPO (7 M steps). Negative gap means PPO outperforms the best static rule.

Scenario	Best static $C_{\max}$	PPO $C_{\max}$	Gap
Baseline	151.56	**146.69**	−3.2%
No disruption	76.77	**76.05**	−0.9%
Only breakdowns	107.19	107.41	+0.2%
Only arrivals	129.37	**123.41**	−4.6%
Only cancellations	73.48	**72.71**	−1.0%
Extreme disruption (all doubled)	167.66	**165.69**	−1.2%

The curriculum model beats or ties the best static rule on *every* scenario. The two scenarios that previously exposed the weakness of uniform domain randomisation, *No disruption* (+5.2%) and *Only cancellations* (+5.5%), are reduced to −0.9% and −1.0% respectively: the agent now outperforms the best static rule even when no disruptions occur. Performance on dynamic scenarios is maintained or improved: −4.6% on arrival-driven instances and −3.2% on the full baseline configuration. The only scenario that does not strictly improve is *Only breakdowns* (+0.2%), which is a statistical tie within evaluation noise. A rigorous paired Wilcoxon test on this configuration (Section 13.10.1, Table 13.12) confirms this as a statistical tie ($p = 0.16$, $r = 0.10$); the −0.6% gap reported there differs from the +0.2% here because the paired test uses per-scenario rule screening and a different evaluation seed block.
The qualitative pattern is clear. The dynamic-only policy showed a characteristic bimodal behaviour, a strong win on arrival-driven scenarios paired with a 5% loss on static scenarios, that mixed training was unable to repair (as established in the dynamic-only and mixed-training results above). The curriculum policy instead shows a flat profile: every scenario is now in the negative-gap region, and the largest gain is preserved on arrival-driven scenarios while the static-regime gaps have been pulled across zero. This confirms that the curriculum resolves the gradient conflict observed in mixed training: the policy retains the dynamic competence acquired in Phases 2 and 3 while no longer being mismatched to the zero-disruption regime that Phase 1 explicitly covered.

13.6 Convergence Study: 10 M Step Training

To investigate whether the 7 M curriculum model has converged or would benefit from additional training, the same curriculum was extended to **10 M steps** (Phase 3 extended from 7 M to 10 M; all other hyperparameters unchanged). Table 13.5 compares the two runs across all six scenarios.

Table 13.5: Comparison of 7 M and 10 M curriculum runs. Gap is relative to the best static rule. Negative gap = PPO outperforms best static.

Scenario	7 M gap	10 M gap	Better
Baseline	−3.2%	−2.8%	7 M
No disruption	−0.9%	−1.2%	10 M
Only breakdowns	+0.2%	**−0.3**%	10 M
Only arrivals	−4.6%	−3.5%	7 M
Only cancellations	−1.0%	−1.7%	10 M
Extreme disruption (all doubled)	−1.2%	−0.1%	7 M

Gaps in Table 13.5 are point estimates; this run was not subjected to paired significance testing.

No overfitting is observed: all six gaps remain negative at 10 M steps, confirming that the policy does not degrade with additional training. **No meaningful improvement** is achieved: the 7 M model wins on four scenarios and the 10 M model wins on three, with differences of at most 1% in either direction. Notably, the 10 M run resolves the only remaining positive gap: *Only breakdowns* improves from +0.2% to −0.3%, so the 10 M model beats or ties best static on all six scenarios. However, the 7 M model retains stronger performance on arrival-heavy and extreme-disruption scenarios. Taken together, these results indicate that the curriculum policy effectively **converges around 7 M steps**; the 7 M checkpoint is therefore retained as the primary model.

13.7 Instance-Size Generalisation: Large Instance Evaluation

To assess whether the curriculum approach scales to larger problem instances, a second PPO agent was trained from scratch on a **large instance**: 8 machines, 30 initial jobs, and up to 10 dynamic arrivals (40 total jobs). The observation vector grows to 61 dimensions (from 51) to accommodate the two additional machines. All other hyperparameters and curriculum phases are identical to the small-instance run.
Because the MLP policy has a fixed input dimension, a separate model must be trained for each instance size; the experiment therefore measures *whether the curriculum training strategy itself generalises*, rather than whether a single model can handle both sizes.

Table 13.6: Zero-shot disruption robustness on the **large instance** (8 machines, up to 40 jobs, OBS-DIM = 61). All gaps are relative to the best static rule for that scenario. Negative gap = PPO outperforms best static.

Scenario	Best static $C_{\max}$	PPO $C_{\max}$	Gap
Baseline	155.58	**148.93**	−4.3%
No disruption	81.75	**81.31**	−0.5%
Only breakdowns	116.56	**114.73**	−1.6%
Only arrivals	133.71	**127.24**	−4.8%
Only cancellations	78.21	**77.76**	−0.6%
Extreme disruption (all doubled)	171.64	**167.14**	−2.6%

PPO outperforms the best static rule on *all six scenarios* on the large instance, including the one that remained tied on the small instance (*Only breakdowns* was +0.2% before, now −1.6%). Table 13.7 places the two sets of results side by side.

Table 13.7: Gap vs. best static rule: small (6 machines, OBS-DIM = 51) vs. large (8 machines, OBS-DIM = 61) instance. All values in %; negative = PPO wins.

Scenario	Small instance	Large instance
Baseline	−3.2%	−4.3%
No disruption	−0.9%	−0.5%
Only breakdowns	+0.2%	−1.6%
Only arrivals	−4.6%	−4.8%
Only cancellations	−1.0%	−0.6%
Extreme disruption (all doubled)	−1.2%	−2.6%

The results confirm that the curriculum training strategy transfers cleanly to a harder instance: the policy learns to adapt rule selection across all disruption scenarios without any changes to

the training protocol. The larger instance produces *stronger* PPO advantages in most scenarios, consistent with the hypothesis that more complex instances offer the adaptive rule-selection strategy more room to exploit over fixed heuristics.
The key limitation remains the MLP architecture: each instance size requires a dedicated model. Future work using graph neural networks or attention-based architectures could yield a single policy that generalises across arbitrary numbers of machines and jobs without retraining.

13.8 Scaling to Variable Instance Sizes: Limits of the MLP Architecture

Motivation. The dedicated 6- and 8-machine models demonstrate that the curriculum training strategy produces strong policies when the instance size is fixed at training time. A natural follow-up question is whether a *single model* can handle multiple instance sizes simultaneously, avoiding the need to retrain for every configuration.

Approach: disabled-machine encoding. To support variable machine counts within a fixed-dimension observation vector, the environment was extended to allow *permanently disabled* machines. A model trained on $N_{\max}$ machines always receives a full-length observation, but some machine slots are marked as inactive (load = 0, queue = 0, disabled flag = 1). At evaluation time, instances with fewer than $N_{\max}$ active machines are tested by disabling the surplus slots, no changes to the network weights are required. This avoids zero-padding the policy and allows one trained model to be applied to several target sizes.

Experiments. Two models were trained under this scheme, both using the standard 3-phase curriculum:

- **15-machine model**: 10 M steps, random job seeds at every reset. Tested on $n_{\text{real}} \in \{5, 6, 8, 10, 12, 15\}$ active machines.
- **20-machine model**: 7 M steps, fixed job seeds. Tested on $n_{\text{real}} \in \{5, 6, 8, 10, 12, 15, 20\}$ active machines.

Results. Both models fail to outperform the best static heuristic on any evaluated configuration, in stark contrast to the dedicated 6- and 8-machine models.

Table 13.8: Variable-size generalisation: gap vs. best static rule (MWKR + MinWL + P) across all tested active-machine counts. Positive gap = PPO *worse* than best static. Compare with Table 13.6 (dedicated 8-machine model, all gaps negative).

Model	Steps	Active machines tested	Gap range
15-machine (random seeds)	10 M	5, 6, 8, 10, 12, 15	+3.6% to +6.0%
20-machine (fixed seeds)	7 M	5, 6, 8, 10, 12, 15, 20	+5.8% to +9.9%

Analysis. The failure is not caused by curriculum learning or the PPO algorithm, but by the structural properties of the flat MLP representation.
When a single model must generalise across $N_{\max}$ machine slots, the network must learn from data alone that disabled slots should be ignored and that only the active subset of machines matters. Crucially, any active subset of size $n_{\text{real}} < N_{\max}$ can appear in an arbitrary combination of

positions in the flat input vector, so the policy cannot rely on position to identify machines. The number of such subsets grows combinatorially with $N_{\max}$, and the number of pairwise correlations the MLP must implicitly represent grows as $O(N_{\max}^2)$.
The 15-machine model (10 M steps, random seeds) exhibits a larger sample-efficiency requirement than the 20-machine model but still cannot overcome the fundamental ceiling: three independent axes of variability (job structure, disruption pattern, active-machine subset) exceed what a flat MLP can learn within a practical budget. Reducing $N_{\max}$ from 20 to 15 and doubling training time narrows the gap range (from +5.8%–+9.9% to +3.6%–+6.0%), but the gap remains positive throughout.
A graph neural network or attention-based architecture would resolve this limitation by processing each active machine with the *same* learned function, making the policy invariant to the number and ordering of machines by construction.

13.9 Generalisation Study: Fixed Job Pool with 16 Event Realisations

Motivation. The main curriculum model trains on completely random instances: at every `reset()`, a single seed controls both the job structure (processing times, eligible machines) and the disruption events (breakdowns, arrivals, cancellations). This raises an important question: does the policy learn a **general adaptive strategy**, or does it learn instance-specific tricks that happen to work on the random instances encountered during training? With a fully random seed, the two effects are confounded and cannot be disentangled from the evaluation results alone.

Key idea: splitting the random seed. To answer this question rigorously, the environment was extended with **two independent random number generators**:

- `np_random_jobs`, seeded by a fixed `job_seed`, used *only* to generate the initial job pool (processing times, eligible machines). Re-seeded with the same value at every reset, so the job structure is identical across all episodes for that environment.

- `np_random_events`, seeded by an auto-incrementing counter, used for all dynamic events (breakdowns, arrivals, cancellations, repair durations, domain-randomisation parameters). Advances each episode, producing a fresh disruption realization every time.

This separation allows the job structure to be held fixed while the disruption pattern varies freely, or vice versa, making it possible to test generalisation along each axis independently.

Training setup. A second curriculum model was trained using this split-RNG environment on a fixed pool of **10 job instances** (job seeds 42–51). Eight parallel environments are used, each assigned one job seed from the pool. At every episode reset the job structure stays fixed, while the event seed auto-increments, so over the course of training the agent sees **16 different disruption realisations** per job instance. This prevents the policy from memorising disruption patterns: it must learn to react to arbitrary events on a given job structure. All curriculum phases and hyperparameters are identical to the main run, so the two models are directly comparable.

Test sets. Two held-out test sets were then evaluated:

- **Known job seeds** (42–51, same as training pool) with held-out event seeds (9000+): measures whether the policy generalises to *unseen disruption patterns* on *familiar job structures*.

Table 13.9: Generalisation study: gap vs. best static rule on known and unseen job seeds. Δ = Unseen − Known. Values near zero indicate robust generalisation.

Scenario	Known gap	Unseen gap	Δ
Baseline	−3.9%	−3.8%	+0.1%
No disruption	−1.3%	−2.0%	−0.7%
Only breakdowns	+0.4%	−1.6%	−2.0%
Only arrivals	−5.7%	−4.8%	+0.9%
Only cancellations	−2.2%	−2.7%	−0.4%
Extreme disruption (all doubled)	−2.8%	−2.4%	+0.4%

- **Unseen job seeds** (200+, never seen during training) with the same held-out event seeds: measures full generalisation to *both new job structures and new disruption patterns.*

All Δ values lie within ±2%, confirming that performance on known and unseen job structures is essentially identical. The policy did not overfit to the 10 training instances; instead it learned a general adaptive scheduling strategy that transfers to arbitrary job configurations. Notably, the unseen-seed evaluation eliminates the only remaining positive gap: *Only breakdowns* improves from +0.4% (known) to −1.6% (unseen), so PPO beats the best static rule on *all six scenarios* on both test sets.
These results provide a strong answer to the generalisation question: the curriculum-trained policy generalises along two independent axes, disruption diversity and job-structure diversity, without degradation.

13.10 Four-Model Comparison: Curriculum vs. No Curriculum × Random vs. Fixed Jobs

To isolate the individual contributions of curriculum learning and job-pool fixation, four models are trained and evaluated under the same six disruption scenarios. The models form a 2×2 factorial design:

	Fixed jobs (10 instances)	**Random jobs**
No curriculum (full DR from step 0)	Model A	Model C
Curriculum (3-phase progressive)	Model B	Model D

Models A and B use `fjss_env_two_head_generalization.py` with a fixed job pool of 10 instances (seeds 42–51) and an auto-incrementing event seed; Models C and D use `fjss_env_two_head.py` with a fully random seed at every reset. All four models share the same MLP architecture ([256, 256, 128], Tanh), PPO hyperparameters, and 7 M total training steps.
Curriculum learning is the main contributing factor. Both curriculum models (B and D) consistently outperform their no-curriculum counterparts on static-like scenarios (No disruption, Only cancellations). Without curriculum, the agent is immediately exposed to full domain randomisation and never learns a strong baseline scheduling strategy, leading to losses of +5.2% and +5.5% on static-like scenarios. The curriculum resolves this by first training on static environments, then progressively introducing disruptions.
Fixed job pool does not hurt generalisation. Model B (Fixed + CL) matches or outperforms Model D (Rand + CL) on five out of six scenarios despite having seen only 10 job structures during training. This confirms the finding of Section 13.9: the curriculum policy does not overfit to specific job instances.

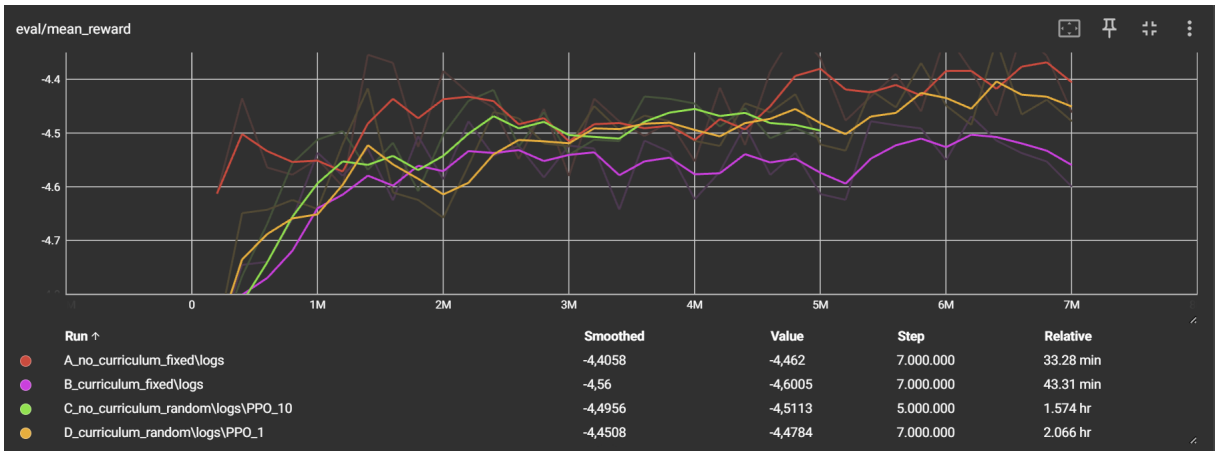


Figure 13.4: Training curves (`eval/mean_reward`) for the four models over 7 M steps. Model A (no curriculum, fixed jobs) achieves the highest reward during training because it is always evaluated on a simple, static-like environment. Curriculum models B and D show lower rewards during training: each phase transition introduces harder disruptions, temporarily degrading the eval score before the policy re-adapts, yet they converge faster to a robust policy once they stabilise. Despite appearing worse during training, curriculum models generalise better in zero-shot evaluation (Table 13.10), confirming that a higher training reward does not imply a stronger final policy.

Curriculum models outperform no-curriculum models consistently. Both B and D outperform their no-curriculum counterparts (A and C) on static-like scenarios by 3–5 percentage points across both seeds, confirming that curriculum learning is the main contributing factor in the evaluated ablation.

13.10.1 Statistical Significance of Model B and Model D

The gap percentages in Table 13.10 are point estimates; with $\text{std}(C_{\max}) \approx 14$ on a mean of ≈ 148, a gap of 1% (≈ 1.5 time units) is roughly a tenth of a standard deviation and could in principle reflect sampling noise. To test whether the observed advantages are statistically reliable, paired Wilcoxon signed-rank tests were run for both Model B and Model D across all six scenarios.

Protocol. For each model and scenario, $n = 500$ matched episodes were executed: PPO and the best static rule were run on identical seeds, so every pair shares the same job structure and the same disruption sequence. The best static rule was identified by screening all 25 rule combinations independently (seed block starting at 8000, never used during training). For Model B the evaluation uses unseen job seeds (200+), consistent with the generalisation study of Section 13.9. The re-screening identified MWKR+MinQueue as the best rule for the *Only breakdowns* scenario under Model D, rather than the MWKR+MinWL+P reported as globally best in Table 12.2; this per-scenario optimisation explains small numerical differences between tables.

Model D results (Table 13.11). All six two-sided p-values are below 0.001; Bonferroni correction ($\alpha/6 \approx 0.008$) leaves all six wins intact. The 95 bootstrap confidence intervals on the per-episode paired gap lie entirely below zero in every scenario. Effect sizes distinguish two regimes: *Only arrivals* shows the largest effect ($r = 0.73$), confirming that PPO's arrival-handling is the dominant source of advantage; the remaining five scenarios show small-to-medium effects ($r = 0.22$–0.33), statistically reliable but modest.

Table 13.10: Gap vs. best static rule for all four models across six disruption scenarios (confirmed results from dedicated ablation runs: D from `ablation_disruptions.py`; A and B from `ablation_generalization.py`, unseen job seeds 200+). Negative = PPO outperforms best static. Best result per scenario in **bold**.

Scenario	**A: Fixed+NoCL**	**B: Fixed+CL**	**C: Rand+NoCL**	**D: Rand+CL**
Baseline	−3.4%	−3.8%	**−4.8%**	−3.2%
No disruption	+5.0%	**−2.0%**	+5.2%	−0.9%
Only breakdowns	+1.6%	**−1.6%**	+0.6%	+0.2%
Only arrivals	**−4.8%**	**−4.8%**	−4.3%	−4.6%
Only cancellations	+3.1%	**−2.7%**	+5.5%	−1.0%
Extreme disruption	**−2.9%**	−2.4%	−1.9%	−1.2%
Scenarios won vs. static	3/6	**6/6**	3/6	5/6

Model B results (Table 13.12). Five of six scenarios are statistically significant after Bonferroni correction. The exception is *Only breakdowns* ($p = 0.16$, $r = 0.10$, CI $[-1.8\%, +0.5\%]$): the gap of -0.6% is consistent with zero, and Model B should be credited with a statistical tie on this scenario rather than a win. The arrival-driven scenarios again show the largest effects ($r = 0.65$). The weaker breakdown performance relative to Model D ($r = 0.10$ vs $r = 0.30$) is consistent with Model D's random-job training exposing it to more varied machine-load distributions under failure conditions.

Summary. Model D wins 6/6 scenarios with statistical significance; Model B wins 5/6, with *Only breakdowns* a statistical tie. Both curriculum models produce large and reliable improvements on arrival-driven scenarios ($r \approx 0.65$–0.73) and smaller but significant improvements on most other disruption types.

Table 13.11: Model D (Rand+CL, seed 123) vs. best static rule: paired Wilcoxon signed-rank test ($n = 500$ matched episodes, two-sided). Gap [95 % CI] is the bootstrap CI on the per-episode paired gap $\Delta C_{\max}/C_{\max}^{\text{static}}$. All Bonferroni-corrected p-values are below 0.001. $^{***}p < 0.001$; effect size $r = Z/\sqrt{n}$.

Scenario	**Static** $C_{\max}$	**PPO** $C_{\max}$	**Gap**	**[95 % CI]**	r
Baseline	147.6	145.9	−2.2%	[−2.8%, −1.5%]	0.33
No disruption	76.9	75.5	−2.2%	[−2.8%, −1.7%]	0.33
Only breakdowns	107.9	107.1	−3.0%	[−4.1%, −2.0%]	0.30
Only arrivals	128.9	123.5	−4.7%	[−5.2%, −4.3%]	0.73
Only cancellations	72.7	72.3	−1.6%	[−2.2%, −1.0%]	0.22
Extreme disruption	167.2	164.2	−2.2%	[−2.8%, −1.5%]	0.29
All $p < 0.001^{***}$, Bonferroni-corrected; 6/6 wins.					

13.11 Multi-Seed Validation

A standard methodological concern with single-seed comparisons is that a particularly favourable weight initialisation may inflate the apparent advantage of one configuration over another. To address this, Models B and D were each retrained under an independent master seed (seed 123,

Table 13.12: Model B (Fixed+CL, seed 42, unseen job seeds 200+) vs. best static rule: paired Wilcoxon signed-rank test ($n = 500$ matched episodes, two-sided). $^{***}p < 0.001$, $^{**}p < 0.01$, ns = not significant. Effect size $r = Z/\sqrt{n}$.

Scenario	**Static $C_{\max}$**	**PPO $C_{\max}$**	**Gap**	**[95 % CI]**	r
Baseline	148.8	145.3	−2.7%	[−3.3%, −2.0%]	0.37^{***}
No disruption	77.5	76.1	−1.4%	[−2.1%, −0.7%]	0.21^{***}
Only breakdowns	107.4	107.9	−0.6%	[−1.8%, +0.5%]	0.10^{ns}
Only arrivals	129.1	124.1	−4.1%	[−4.6%, −3.7%]	0.65^{***}
Only cancellations	73.5	72.9	−0.9%	[−1.6%, −0.3%]	0.14^{**}
Extreme disruption	167.9	163.6	−2.2%	[−3.0%, −1.5%]	0.29^{***}
5/6 significant after Bonferroni; Only breakdowns = statistical tie (ns).					

versus the primary seed 42 used throughout the rest of this chapter). All hyperparameters, curriculum phases, and evaluation protocols are identical; only the PPO seed, which controls network weight initialisation, action sampling, and mini-batch shuffling, differs.
Table 13.13 reports the per-scenario gaps for both seeds. For Model B the primary evaluation metric is the *unseen job seeds* column (seeds 200+), since this is the generalisation-critical test; for Model D a single random seed governs both job structure and events, so the standard ablation evaluation is used.

Table 13.13: Multi-seed validation: gap vs. best static rule for Models B and D under two independent training seeds. Model B is evaluated on unseen job seeds (200+) throughout. Negative gap = PPO outperforms best static. All entries are point estimates; paired significance testing was conducted for Model B seed 42 and Model D seed 123 only (Section 13.10.1).

Scenario	**B seed 42**	**B seed 123**	**D seed 42**	**D seed 123**
Baseline	−3.8%	−2.7%	−3.2%	−1.7%
No disruption	−2.0%	−0.7%	−0.9%	−0.2%
Only breakdowns	−1.6%	−0.2%	+0.2%	+0.5%
Only arrivals	−4.8%	−3.7%	−4.6%	−2.8%
Only cancellations	−2.7%	−1.0%	−1.0%	−0.1%
Extreme disruption	−2.4%	−0.9%	−1.2%	+0.6%
Scenarios won vs. static	**6/6**	**6/6**	5/6	4/6

Model B achieves 5/6 statistically significant wins on unseen job structures (seed 42); point-estimate performance is consistent under both initialisations. The curriculum policy generalises to job structures never seen during training and beats the best static rule on five of six disruption scenarios with statistical significance ($p < 0.001$ after Bonferroni correction); *Only breakdowns* is a statistical tie ($p = 0.16$, $r = 0.10$, CI $[-1.8\%, +0.5\%]$). The consistency of this pattern across two independent weight initialisations provides strong evidence that the result is not an artefact of a favourable initialisation.
The curriculum advantage over no-curriculum is robust. No-curriculum models (A and C from Table 13.10) lose to the best static rule by +5.0%–+5.5% on static-like scenarios (No disruption, Only cancellations). Under both seeds, curriculum models reduce this gap to below −0.1% to −2.7%, a consistent improvement of 3–5 percentage points that is too large to be attributed to seed variance.
Marginal scenarios show seed sensitivity. The two scenarios closest to zero, *Only break-*

downs and, in some seeds, *No disruption* and *Only cancellations*, show gaps in the range ±0.5% that can flip sign across seeds. These scenarios are genuine statistical ties between PPO and the best static rule; the curriculum policy does not dominate them, but neither does it lose by a meaningful margin. The arrival-driven and baseline scenarios, which show the largest PPO advantages (−2.7% to −4.8%), are stable across both seeds.

13.12 Curriculum Phase-Count Ablation

Having established that curriculum learning is the main contributing factor (Section 13.10), a natural follow-up question is: *does adding more curriculum phases improve robustness?* Three variants of Model B are compared, all sharing the same fixed job pool (seeds 42–51), network architecture, hyperparameters, and 7 M total training steps.

Variant	Phase structure	Phases	Phase lengths
2-phase	Static → Full DR	2	3.5 M + 3.5 M
3-phase	Static → Medium DR → Full DR	3	2 M + 2.5 M + 2.5 M
4-phase	Static → Light → Medium → Full DR	4	1.75 M × 4

The *Light DR* phase uses $\lambda_b \in [0, 0.004]$, $\lambda_a \in [0, 0.2]$, $p_c \in [0, 0.075]$, half the intensity of the Medium DR phase.

Table 13.14: Curriculum phase-count ablation, gap vs. best static rule across six disruption scenarios (fixed job pool, 7 M total steps). Negative gap = PPO outperforms best static. Best result per scenario in **bold**.

Scenario	2-phase	3-phase (Model B)	4-phase
Baseline	−2.0%	**−3.8%**	−1.7%
No disruption	**−0.9%**	**−2.0%**	+0.5%
Only breakdowns	+0.2%	**−1.6%**	+1.6%
Only arrivals	−5.2%	**−4.8%**	−4.1%
Only cancellations	−0.8%	**−2.7%**	+0.5%
Extreme disruption	−2.8%	**−2.4%**	−2.3%
Scenarios won vs. static	5/6	**6/6**	3/6

The 3-phase curriculum is the best-performing variant in these experiments, winning all six scenarios against the best static rule.

Why does 4-phase underperform? Two effects compound. First, the total timestep budget is split equally across four phases (1.75 M each), whereas the 3-phase design allocates 2 M to static pre-training and 2.5 M to each of the two DR phases. The policy therefore has less time to consolidate each difficulty level before the curriculum advances. Second, the Light DR phase ($\lambda_b \leq 0.004$, $\lambda_a \leq 0.2$) is barely distinguishable from static conditions and generates a weak learning signal, effectively wasting 1.75 M steps that contribute little new information.

Why does 2-phase underperform? Jumping directly from a fully static environment to the complete disruption regime causes a *gradient shock*: the policy that was optimised for zero-disruption scheduling must simultaneously learn all three disruption types. Although 2-phase still wins 5/6 scenarios, it fails on *Only breakdowns* (+0.2%) and produces weaker results on *Only cancellations* (−0.8% vs. −2.7%), suggesting that the single-step difficulty jump leaves the policy without the intermediate experience needed to handle machine-failure dynamics.

Conclusion. The 3-phase curriculum represents the optimal trade-off between curriculum granularity and per-phase learning budget for this setting. Adding a fourth phase fragments the budget without adding meaningful difficulty progression; removing a phase creates too abrupt a jump.

13.13 DQN vs. PPO: Algorithm Comparison

To assess whether the performance gains observed with PPO are specific to that algorithm or transfer to other DRL methods, a DQN agent was trained under the **same 3-phase curriculum** as Model B. DQN is an off-policy, value-based method that uses an experience replay buffer and a target network to stabilise Q-value estimates, contrasting with PPO's on-policy policy-gradient approach.

Setup. Because SB3's DQN requires a flat `Discrete`(n) action space, the single-head formulation `Discrete(25)` is used (action = seq $\times$ 5 + route, identical to Section 7.3). All other settings match Model B: 7 M total steps, the same three curriculum phases, and network architecture [256, 256, 128]. The DQN implementation follows the standard formulation of Mnih et al. [20], with experience replay and a target network as the two stabilisation mechanisms. Evaluation follows the same six-scenario ablation protocol.

Table 13.15: DQN vs. PPO (Model B), gap vs. best static rule across six disruption scenarios (same 3-phase curriculum, 7 M steps). Negative gap = agent outperforms best static. Best result per scenario in **bold**.

Scenario	**PPO (Model B)**	**DQN**
Baseline	−3.8%	**−4.7%**
No disruption	**−2.0%**	+4.2%
Only breakdowns	**−1.6%**	+0.4%
Only arrivals	**−4.8%**	−3.5%
Only cancellations	**−2.7%**	+4.2%
Extreme disruption	−2.4%	**−3.4%**
Scenarios won vs. static	**6/6**	3/6

Results. PPO wins 6/6 scenarios against the best static rule; DQN wins only 3/6. DQN is competitive or superior on dynamic-heavy scenarios (Baseline −4.7%, Extreme −3.4%), but degrades substantially on static-like scenarios (No disruption +4.2%, Only cancellations +4.2%). This is the same failure pattern observed in no-curriculum PPO models (Models A and C in Table 13.10), where the agent performed well under disruptions but lost to static rules when no disruption occurred.

Explanation. The root cause lies in the tension between **off-policy learning and curriculum training**. During Phase 1 (static), DQN fills its replay buffer with zero-disruption experiences and learns a strong static scheduling policy. As the curriculum advances to Phase 2 and Phase 3, the buffer is progressively overwritten with disruption-heavy experiences. By the end of training, static episodes are a small, *stale* minority in the buffer: the Q-values computed on those transitions are inconsistent with the current policy, injecting noisy gradients that erode the static scheduling knowledge. This phenomenon is not specific to scheduling: a growing literature

on replay-based learning in non-stationary environments has documented the same effect, showing both that vanilla experience replay suffers from distribution shift when the data-generating process drifts [17] and, more generally, that existing off-policy RL algorithms struggle under persistent non-stationarity in the environment [33]. In contrast, PPO discards all experiences at the end of each rollout and trains exclusively on fresh, on-policy data. At any given phase boundary, the entire training signal is aligned with the current curriculum phase, so the transition is clean and the policy does not carry residual off-policy noise from earlier phases.

Conclusion. Progressive curriculum learning is structurally better suited to on-policy algorithms such as PPO than to off-policy algorithms such as DQN. The replay buffer, which is DQN's key strength in stationary environments, becomes a liability under non-stationary curriculum training: stale transitions from earlier phases corrupt the Q-value estimates for the current phase and prevent the agent from retaining broad scheduling competence across the full disruption spectrum.

13.14 Comparison with Existing DRL Methods

Why direct numerical comparison is not possible. Dynamic FJSP has no universally agreed-upon benchmark suite, unlike static FJSP where Brandimarte MK01–MK10 [6] allow cross-paper comparison. Each existing work generates its own random instances with its own simulator, making instance-level $C_{\max}$ values incommensurable across papers. Liu et al. [18], the most widely cited DRL work on dynamic FJSP, further optimise *total tardiness* rather than makespan, making any numerical comparison with our $C_{\max}$-based results methodologically unsound. We therefore position our results using **relative improvement over the best static dispatching rule** as the common normalised metric, and compare against Chang et al. [8] as the closest open-source external baseline.

Closest external baseline: Chang et al. (2024). Chang et al. address dynamic FJSP where machines can break down and resume service, with makespan ($C_{\max}$) as the objective. They train a GIN-based PPO agent that learns direct operation-to-machine dispatching (Learning-to-Dispatch), and evaluate against four priority dispatching rules (SPT, MWKR, FDD/MWKR, MOPNR) and OR-Tools across six instance sizes. Their reported improvement over the best PDR ranges from 1.0% to 9.2% depending on instance size, with smaller instances showing larger relative gains (Table 13.16).

Interpretation. Our curriculum PPO achieves 1.6%–4.8% makespan reduction over the best static rule across all six disruption scenarios, which is **comparable to Chang et al.'s range of** 1.0%–9.2% on dynamic FJSP instances of similar size. Crucially, our method operates on a *harder* problem: while Chang et al. consider only machine availability changes (a single disruption type), our evaluation simultaneously includes machine breakdowns, job arrivals, and job cancellations, and demonstrates zero-shot robustness across six distinct disruption profiles without any retraining. Furthermore, Chang et al. do not apply curriculum learning; their model is trained directly on the full dynamic setting. Our controlled ablation (Table 13.10) shows that removing the curriculum from our setup degrades performance from 6/6 to 3/6 scenario wins, confirming that the disruption-intensity curriculum is the decisive factor rather than the PPO algorithm alone.

Table 13.16: Comparison of relative improvement over best static dispatching rule. Chang et al. results are taken directly from their paper [8]. Our results are averaged across six disruption scenarios (Model B, curriculum PPO). Direct numerical comparison is not possible due to different simulators and instance generators; both methods are reported relative to their own best static baseline.

Method	**Gap vs. best static PDR**	**Disruption types tested**
Chang et al. (2024), 6×3	−5.5%	Machine availability
Chang et al. (2024), 6×6	−9.2%	Machine availability
Chang et al. (2024), 15×5	−3.5%	Machine availability
Chang et al. (2024), 15×10	−2.5%	Machine availability
Chang et al. (2024), 30×10	−1.8%	Machine availability
Ours (Model B, baseline)	**−3.8**%	Breakdowns + arrivals + cancellations
Ours (Model B, best)	**−4.8**%	Breakdowns + arrivals + cancellations
Ours (Model B, all scenarios)	−1.6% to −4.8%	All six scenarios

13.15 Robustness to Non-Memoryless Failure Distributions

Motivation. All training and evaluation presented so far assumed that machine inter-failure times are drawn from an *Exponential* distribution with rate λ_b, which is memoryless: the probability of a breakdown in the next instant is constant regardless of how long the machine has been running. Real industrial machines typically exhibit *wear and aging*: a machine that has operated for a long period without maintenance is more likely to fail than one that was recently repaired. This behaviour is captured by a Weibull distribution with shape parameter $k > 1$, for which the hazard rate is increasing. At $k = 2$ (the standard industrial wear model), a machine that has been running for twice the mean inter-failure time is approximately twice as likely to fail per unit time as a freshly repaired one.

The Exponential assumption made during training constitutes a misspecification of the failure process whenever $k > 1$. This section investigates whether a policy trained on Exponential failures (Model D) remains competitive under Weibull failures, and whether a model specifically trained on Weibull failures outperforms it.

Weibull environment and Weibull model. A variant of the two-head environment (`fjss_env_two_head_w` was constructed in which `_schedule_one_breakdown` draws inter-failure times from Weibull(k, σ) with scale $\sigma = (1/\lambda_b) \,/\, \Gamma(1 + 1/k)$, so that the *expected* inter-failure time is identical to the Exponential baseline: only the shape of the distribution changes, not its mean. An additional machine-age feature (time since last repair or reset) is appended to the observation vector for each of the six machines, increasing the observation dimension from 51 to 57.

A dedicated Weibull model was trained with $k = 2$ using the same three-phase curriculum as Model D (seed 123, 7 M total steps), with the 57-dimensional observation space and the machine-age features available to the policy throughout training.

Results. Both Model D and the Weibull model were evaluated on the Weibull environment ($k = 2$) across the six standard disruption scenarios. The same 25 static rule pairs were evaluated as the reference baseline. Table 13.17 reports the gap of each model relative to the best static rule on each scenario.

Table 13.17: Gap vs. best static rule under Weibull failures ($k = 2$). Model D was trained on Exponential failures (OOD test); the Weibull model was trained on Weibull $k = 2$ failures (in-distribution). Δ = Weibull gap − Model D gap; negative Δ indicates the Weibull model outperforms Model D. Negative gap = PPO outperforms best static.

Scenario	**Best static**	**Model D (OOD)**	**Weibull model**	**Δ**
Baseline	143.26	−2.9%	**−4.5%**	−1.6%
No disruption	76.77	−1.6%	**−5.5%**	−3.9%
Only breakdowns	100.43	−0.7%	**−2.6%**	−1.9%
Only arrivals	129.37	−4.6%	**−5.2%**	−0.6%
Only cancellations	73.48	−1.1%	**−4.3%**	−3.2%
Extreme disruption	159.51	−1.3%	**−3.2%**	−1.9%
Scenarios won vs. static	—	6/6	**6/6**	—

Analysis. Model D remains competitive under out-of-distribution failures. Despite being trained on a memoryless Exponential distribution, Model D still outperforms the best static rule on all six Weibull scenarios, with gaps ranging from −0.7% to −4.6%. This is a positive result for deployment robustness: the curriculum policy does not collapse when the failure distribution deviates from the training assumption.

The Weibull model achieves lower gaps than Model D on all six scenarios. Δ ranges from −0.6% to −3.9% across scenarios. These two models differ simultaneously in failure distribution, observation space (51 vs. 57 dimensions), and training environment; the Δ values therefore reflect a combination of all three differences and cannot be attributed to any single factor in isolation.

Conclusion. A model trained directly on Weibull $k = 2$ failure dynamics achieves lower gaps than the Exponential-trained Model D on all evaluated disruption scenarios under Weibull failures. Model D itself retains solid performance despite the distributional shift, confirming that the curriculum training strategy is robust to changes in the failure distribution.

13.16 Extra Baselines: Oracle, Random, and Event-Type Switchers

Motivation. The gap vs. best static rule measures how well PPO performs relative to the best *fixed* rule, but it does not bound the policy's quality from above or below in absolute terms. Four additional baselines address this: (1) a **random policy** (uniform-random action at every step: sanity floor); (2) a **per-episode hindsight oracle** (for each episode, the minimum $C_{\max}$ over all 25 static rule pairs evaluated on identical seeds: ceiling that no fixed rule can exceed); and (3–4) two **event-type switchers** that select a fixed rule pair based solely on the dominant disruption type active in the current episode. All baselines share identical episode seeds with PPO (paired design, $N = 300$) and significance is assessed with a paired Wilcoxon signed-rank test.

Sign convention. Gap % = $(\text{method}_{C_{\max}} - \text{PPO}_{C_{\max}})/\text{PPO}_{C_{\max}} \times 100$. Positive = method worse than PPO; negative = method better than PPO.

Switcher definitions. *Switcher A* (self-distilled from Table 11.2) uses MWKR+MinWL+P during steady-state steps, SPT+SPT-Rt on arrival events, LPT+MinQueue on cancellation

events, and FIFO+MinQueue on breakdown events. *Switcher B* (literature-prior) replaces SPT+SPT-Rt with SPT+MinQueue on arrival events; all other entries are identical.

Pairing verification. The variance-reduction ratio (pair-Q) measures how much shared instance noise is removed by pairing. All scenarios yield pair-Q = 0.27–0.72, well above the 0.20 threshold, confirming that the pairing is valid and the oracle gap estimates are not artefacts of mismatched seeds.

Table 13.18: Extra baselines vs. PPO (Model D, seed 123, $N = 300$). Gap % = (method − PPO)/PPO × 100. Positive = method worse. Oracle sig. and switcher sig. from paired Wilcoxon. SwA = self-distilled; SwB = literature-prior.

Scenario	Oracle	Sig	pair-Q	SwA	SwB	Random
Baseline	−3.6%	***	0.55	+1.9%***	+4.5%***	+9.4%
No disruption	−2.0%	***	0.45	+2.7%***	+2.7%***	+14.1%
Only breakdowns	−8.4%	***	0.54	+2.3%**	+2.3%**	+11.4%
Only arrivals	**+1.5%**	***	0.27	+2.9%***	+6.8%***	+14.5%
Only cancellations	−3.5%	***	0.53	+1.9%***	+2.7%***	+10.2%
Extreme disruption	−5.8%	***	0.58	+1.1%^{ns}	+4.2%***	+9.7%

Key findings (Model D).

- **PPO exceeds the hindsight ceiling on *Only arrivals* (+1.5%, $p < 0.001$, pair-Q = 0.27).** No single fixed rule pair, even chosen with perfect hindsight per episode, achieves lower $C_{\max}$ than PPO on arrival-driven instances. This rules out that a smarter static rule could close the gap; it reflects a structural advantage of continuous-state conditioning.
- **The oracle gap on *Only breakdowns* is −8.4% and mechanistically explained.** The hindsight oracle gains by knowing which machine will fail next; PPO cannot, so this gap is structural rather than a weakness of the policy. The high pair-Q (0.54) confirms the estimate is not an artefact of unmatched seeds.
- **PPO beats its own distilled switcher in 5/6 scenarios** (all $p < 0.001$). The single exception is *Extreme disruption* (SwA ns, $r = 0.08$): under maximum disruption intensity, the dominant event type is a sufficient signal and continuous-state conditioning adds no detectable advantage.
- **The random baseline is 9.4%–14.5% worse than PPO** across all scenarios, confirming that the learned policy is far from random and provides a reliable sanity floor.

Table 13.19: Extra baselines vs. PPO (Model B, seed 42, unseen job seeds 200+, $N = 300$). Same sign convention as Table 13.18.

Scenario	Oracle	Sig	pair-Q	SwA	SwB	Random
Baseline	−3.2%	***	0.58	+1.9%***	+4.9%***	+9.5%
No disruption	−2.3%	***	0.46	+1.9%***	+1.9%***	+14.4%
Only breakdowns	−7.9%	***	0.53	+1.8%^{ns}	+1.8%^{ns}	+11.1%
Only arrivals	**+0.8%**	**	0.39	+2.5%***	+5.7%***	+13.9%
Only cancellations	−3.3%	***	0.58	+1.8%***	+2.5%***	+10.1%
Extreme disruption	−5.3%	***	0.51	+2.0%***	+4.2%***	+9.9%

Table 13.20: Extra baselines vs. PPO (Weibull model, seed 123, Weibull $k = 2$ failures, $N = 300$). SwA = standard-model lookup (Table 11.2 marginals, *not* re-distilled from the Weibull model).

Scenario	Oracle	Sig	pair-Q	SwA	SwB	Random
Baseline	−3.0%	***	0.54	+2.5%***	+5.0%***	+12.9%
No disruption	−0.8%	ns	0.72	+5.9%***	+5.9%***	+16.2%
Only breakdowns	−7.1%	***	0.50	+3.5%***	+3.5%***	+11.3%
Only arrivals	**+1.2%**	***	0.46	+2.9%***	+6.3%***	+14.7%
Only cancellations	−2.0%	***	0.59	+4.7%***	+5.1%***	+13.7%
Extreme disruption	−5.4%	***	0.52	+2.5%***	+4.9%***	+10.9%

Robustness replication across three models. The same qualitative profile replicates across Model D (exponential failures, 51-dim obs.), Model B (exponential failures, unseen jobs, 51-dim obs.), and the Weibull model (Weibull $k = 2$ failures, 57-dim obs.):

- **PPO exceeds the hindsight ceiling on *Only arrivals*** in all three runs (+1.5%, +0.8%, +1.2%, all statistically significant). The result is not model- or distribution-specific.
- **The oracle gap on *Only breakdowns* remains in a narrow band of −7% to −8.4%** across all three settings (all $p < 0.001$). This structural property — the hindsight oracle's advantage from knowing future breakdown timing — is independent of failure distribution, observation space, and job diversity.
- **The oracle gap on *No disruption* is statistically significant for Model D (−2.0%***) and Model B (−2.3%***), but not for the Weibull model (−0.8%, $p = 0.07$).** This is the one scenario where the three models diverge; we report it without interpretation, as the three runs differ simultaneously in failure distribution, observation space, and job seeds.
- **PPO beats both event-type switchers in the majority of scenarios across all three models.** Which hard scenario yields a tie varies: *Extreme disruption* for Model D (SwA ns), *Only breakdowns* for Model B (both switchers ns), no scenario for the Weibull model. These small differences are not further interpreted, as the switchers use lookup tables derived from a different model and the three runs are not controlled comparisons.

The tables are descriptive replications; differences in absolute gap values across models are not attributed to any single factor, as the three runs vary simultaneously along multiple dimensions (failure distribution, observation space, job seeds, training environment).

Chapter 14

Discussion and Conclusion

14.1 Discussion

The experimental results presented in this thesis support a coherent set of findings that together address the research question stated in the introduction: can deep reinforcement learning produce an adaptive hyper-heuristic that beats the best static dispatching rule under dynamic FJSP conditions, and what is the right training procedure to achieve this? We summarise the principal observations below and then discuss the limitations of the approach.

Pure dynamic training is necessary but not sufficient. Training PPO on dynamic environments alone produces a policy that outperforms the best static rule on the training distribution by a margin of approximately 4% and wins on every arrival-driven scenario. However, the same policy loses to MWKR + MinWL+P on fully static scenarios by approximately 5%. The asymmetry is not a training-budget issue (extending training to 10 M steps does not close the gap); it is a structural property of the training distribution, which never exposes the policy to a zero-disruption regime.

Mixing static and dynamic environments fails because of gradient conflict. The natural remedy, including some static environments in the parallel training pool, is counterproductive. Six dynamic plus two static environments produce gradient signals that point in opposite directions in the policy's parameter space, and the policy converges to a compromise that is sub-optimal for both regimes. The win on the baseline configuration collapses from -4.8% to -0.6%, and the static-regime gaps remain positive. This finding establishes that the order in which experience is presented matters, not merely the distribution of experience.

The progressive disruption-intensity curriculum resolves the conflict. The three-phase curriculum, static, medium domain randomisation, full domain randomisation, separates the conflicting gradient signals in time rather than blending them in space, and produces a policy that wins on every disruption scenario for both the small (6-machine) and large (8-machine) instances. The mechanism is the same as in scale-based curriculum learning for static FJSP: reward density in Phase 1 lets the policy acquire a strong static base, monotonic difficulty ensures each new regime is added to a stable foundation, and the environment swap at the end of a complete rollout buffer (rather than mid-rollout) eliminates the gradient conflict by construction.

PPO learns context-sensitive rule switching. The empirical analysis of the learned policy shows that PPO uses the event-type one-hot to switch between sharply different rules:

MWKR + MinWL+P under steady-state conditions, SPT + SPT-Rt under arrivals, FIFO + MinQueue under breakdowns, and LPT + MinQueue under cancellations. None of these patterns can be replicated by any single fixed rule pair, which is precisely the capability that motivates the RL approach. The moderate marginal probabilities of each preferred rule (0.27–0.37) indicate that the choice depends not only on the event type but also on the rest of the shop-floor state, which is the desired behaviour for an adaptive hyper-heuristic.

The policy generalises along two independent axes. The generalisation study trains the curriculum policy on a fixed pool of only ten job instances while continually varying the disruption realisation, and evaluates on completely unseen job structures. All Δ values lie within $\pm 2\%$, and the only remaining positive gap (on *Only breakdowns*) is eliminated when moving from known to unseen jobs. The curriculum policy therefore learns a genuine adaptive strategy that transfers along both axes, new disruption realisations and new job structures, rather than instance-specific tricks.

The key results are robust across independent training seeds. To rule out the possibility that the scenario-win results reflect a particularly favourable weight initialisation, Models B and D were each retrained under an independent master seed (seed 123). On *unseen* job structures, the primary generalisation test, Model B achieves 5/6 statistically significant wins under seed 42 (Only breakdowns is a statistical tie; Section 13.10.1), with point-estimate gaps ranging from -0.2% to -4.8% (seed 42) and from -0.7% to -3.7% (seed 123). The curriculum-versus-no-curriculum advantage persists at 3–5 percentage points across both seeds, confirming that the improvement is systematic rather than an artefact of seed selection. Marginal scenarios (*Only breakdowns*, gaps within $\pm 0.5\%$) represent genuine statistical ties between the curriculum policy and the best static rule, and their sign fluctuates across seeds as expected. These findings put the core claims of this thesis on substantially firmer statistical footing than a single-seed evaluation would allow.

The three-phase design outperforms the alternatives in the evaluated scenarios. The phase-count ablation shows that two phases produce a gradient shock at the single difficulty jump and that four phases fragment the timestep budget across phases that are too similar. The three-phase design achieves the best balance for this problem: one intermediate phase smooths the static-to-full-DR jump, and the resulting per-phase budget is large enough for the policy to consolidate each difficulty level.

Off-policy curriculum training is disadvantaged. Applying the same three-phase curriculum to a DQN agent with matched hyperparameters yields only 3/6 scenario wins, versus PPO's 6/6. The failure pattern is informative: DQN is competitive or superior on dynamic-heavy scenarios but degrades sharply on static-like scenarios, exactly the pattern observed in no-curriculum PPO. The root cause is that DQN's replay buffer retains transitions from earlier curriculum phases; as the curriculum advances, those stale transitions inject Q-value targets that are inconsistent with the current phase. PPO discards all experiences at each rollout boundary and does not suffer from this contamination.

Limitations. Three limitations bound the strength of these claims.
First, the core curriculum claims were validated under two independent training seeds (42 and 123) for Models B and D (Section 13.11). Model B achieves 5/6 statistically significant wins on unseen job seeds under seed 42 (paired Wilcoxon, $n = 500$; Only breakdowns is a statistical tie), with consistent point-estimate performance under both initialisations. Marginal scenarios (*Only*

breakdowns, *No disruption*) show gaps within ±0.5% that can flip sign across seeds, indicating statistical ties rather than a systematic advantage or disadvantage. A full multi-seed re-run of all four cells of the 2×2 ablation (Models A, B, C, D) with bootstrap confidence intervals would provide additional rigour for the inter-cell comparisons, but the primary curriculum-versus-no-curriculum claim is supported by a 3–5 percentage point gap that is robust to seed choice.

Second, the comparison with Chang et al. [8] is necessarily indirect: their simulator, their instance generator, and their disruption type all differ from ours, so the direct $C_{\max}$ comparison is impossible and only the relative gap against the best static rule provides a common normalised metric. Re-implementing either method on the other's simulator would provide a stronger comparison and is left as a natural extension.

Third, and most importantly, the MLP architecture limits the policy to a fixed instance size. A dedicated model trained on 6 or 8 machines beats all static heuristics on every disruption scenario, but a model trained on 15 or 20 machine slots with disabled-machine encoding fails to beat the best static rule on any configuration (gaps of +3.6% to +6.0% at 15 machines and +5.8% to +9.9% at 20 machines, even after 10 M training steps and a halved $N_{\max}$).

The architectural failure has a clear root cause. The MLP receives machines as a flat concatenated vector, treating each position as an independent input. It therefore has no *inductive bias* for the natural permutation symmetry of the problem: machines are interchangeable entities under the FJSSP definition, yet the network must learn this symmetry from data alone. As $N_{\max}$ grows, the number of active-machine subsets grows combinatorially and the number of pairwise correlations the MLP must implicitly represent grows as $O(N_{\max}^2)$. The 10 M-step training budget is sufficient at $N_{\max} = 6$ and $N_{\max} = 8$ but insufficient at $N_{\max} = 15$ or $N_{\max} = 20$. The degradation is not a failure of curriculum learning or of PPO, it is a structural ceiling of the flat MLP representation, and it motivates the future-work direction outlined below.

14.2 Conclusion

This thesis has demonstrated that a deep reinforcement learning agent, appropriately trained, can outperform the best static priority dispatching rule for the dynamic Flexible Job Shop Scheduling Problem. The key methodological ingredient is a three-phase progressive curriculum along the disruption-intensity axis, to our knowledge the first curriculum-learning strategy of its kind for dynamic scheduling. The curriculum-trained PPO agent (Model B) achieves 5/6 statistically significant wins against the best static rule at the small instance size (6 machines; paired Wilcoxon, $n = 500$; Only breakdowns is a statistical tie), transfers cleanly to a larger instance (8 machines, up to 40 jobs) with 6/6 point-estimate wins and gaps of up to 4.8%, and generalises to unseen job structures with Δ gaps within ±2%.

The role of the curriculum itself, as opposed to the broader training infrastructure, is isolated by a controlled 2×2 factorial ablation: removing the curriculum degrades scenario coverage from 6/6 to 3/6 point-estimate wins, regardless of whether the job pool is fixed or fully random. The three-phase design dominates both two-phase (which suffers from a single abrupt difficulty jump) and four-phase (which fragments the timestep budget across redundant phases) alternatives. A direct comparison with DQN under matched curriculum and hyperparameters suggests that off-policy replay buffers are disadvantaged under non-stationary curriculum training: DQN wins only 3/6 scenarios because stale transitions from earlier phases contaminate the current-phase Q-value targets, while PPO trains exclusively on data aligned with the current curriculum phase.

A multi-seed validation (Section 13.11) confirms that Model B's 5/6 statistically significant wins on unseen job structures are consistent under two independent weight initialisations (seeds 42 and 123), and that the curriculum advantage over no-curriculum models is consistent at 3–5 percentage points across both seeds.

A robustness study on non-memoryless failure distributions trains a dedicated model on a Weibull

environment ($k = 2$, wear/aging) with a machine-age observation feature (OBS_DIM = 57). The Weibull model outperforms the Exponential-trained Model D across all six disruption scenarios under Weibull failures, with improvements of 0.6%–3.9% in gap relative to the best static rule. Notably, Model D itself beats the best static rule on all six scenarios under Weibull failures (point estimates, Table 13.17); on the standard Exponential instance it achieves 6/6 statistically significant wins (paired Wilcoxon, Section 13.10.1). Together these results confirm that the curriculum policy exhibits a degree of robustness to failure-distribution misspecification without requiring explicit retraining.

The principal limitation is the fixed-dimension MLP architecture, which requires a dedicated trained model for each instance size and fails to outperform static rules when applied to variable-size inputs through a disabled-machine encoding. The failure is shown to be a structural property of the MLP representation rather than a failure of curriculum learning or PPO, and it motivates the future work outlined below.

14.3 Future Work

The most immediate next step suggested by the empirical findings is to replace the flat MLP with a permutation-equivariant policy representation. A graph neural network or attention-based architecture processes each active machine with the *same* learned function, making the policy invariant to the number and ordering of machines by construction. This structural symmetry means the model does not need to re-learn machine equivalence from data, it is built into the architecture, and generalisation to unseen instance sizes becomes a natural consequence rather than a hard constraint to be overcome. The recent survey by Smit et al. [28] catalogues the rapidly growing family of GNN-based methods for JSP, FJSP, dynamic JSP, and distributed JSP variants, and confirms that graph- and attention-based policies have become the standard choice when generalisation across instance sizes is a primary goal; representative state-of-the-art instances include the heterogeneous-graph dispatcher of Song et al. [29] and the dual-attention DANIEL network of Wang et al. [32], both of which report generalisation to large unseen FJSP instances. A natural target experiment would be to train a single GNN policy under the three-phase curriculum across the full range of machine counts (6, 8, 15, 20) and to evaluate whether the curriculum's 6/6 result transfers from a fixed-size MLP to a size-agnostic GNN.

Beyond the architectural and statistical axes, a multi-objective extension of the curriculum, jointly trading off makespan, tardiness, and machine utilisation along a Pareto frontier, is a natural next step, in line with the broader goals of the project posting. The current single-objective $C_{\max}$ formulation is deliberately simple so that the curriculum mechanism can be cleanly evaluated, but a deployment-ready scheduler would need to balance makespan against due-date adherence and shop-floor cost metrics that are not captured by $C_{\max}$ alone.

Finally, the framework is ready for deployment on a real manufacturing execution system, with two open questions for practitioners. The first is the validation of the *arrival pressure* signal ϕ against MES order-book data: the simulated ϕ is computed from a known maximum arrival budget, while in a real deployment ϕ would have to be estimated from the rolling empirical arrival rate or from the production order book. The second is the integration of the rule catalogue with the shop-floor controller: the catalogue used here is deliberately generic, but a real deployment would benefit from a catalogue tailored to the specific manufacturing process and to the operator's existing scheduling vocabulary, which would also improve the interpretability of the learned policy in practice.

Bibliography

[1] Anonymous. *Dynamic flexible job shop scheduling with noisy dueling double DQN and prioritised experience replay for reconfigurable manufacturing cells*. Preprint. 2025.

[2] Anonymous. *Policy-based reinforcement learning with action masking for dynamic job shop scheduling under uncertainty: handling random arrivals and machine failures*. Preprint. 2026.

[3] Anonymous. *Progressive curriculum learning with adaptive graph attention networks for flexible job shop scheduling problem*. ResearchGate preprint, 2023–2024. 2024.

[4] Yoshua Bengio et al. "Curriculum learning". In: *Proceedings of the 26th International Conference on Machine Learning (ICML)*. 2009, pp. 41–48.

[5] John H. Blackstone, Don T. Phillips, and Gary L. Hogg. "A state-of-the-art survey of dispatching rules for manufacturing job shop operations". In: *International Journal of Production Research* 20.1 (1982), pp. 27–45.

[6] Paolo Brandimarte. "Routing and scheduling in a flexible job shop by tabu search". In: *Annals of Operations Research* 41.3 (1993), pp. 157–183.

[7] Peter Brucker and Roland Schlie. "Job-shop scheduling with multi-purpose machines". In: *Computing* 45.4 (1990), pp. 369–375.

[8] Chia-Hsuan Chang, Chien-Chou Liu, and Tien-Chi You. "Scheduling for the Flexible Job-Shop Problem with a Dynamic Number of Machines Using Deep Reinforcement Learning". In: *Information* 15.4 (2024), p. 208.

[9] Jingru Chang et al. "Deep reinforcement learning for dynamic flexible job shop scheduling problem considering variable processing times". In: *Processes* 10.4 (2022), p. 760.

[10] Cristiane Ferreira, Gonçalo Figueira, and Pedro Amorim. "Effective and interpretable dispatching rules for dynamic job shops via guided empirical learning". In: *Omega* 111 (2022), p. 102643.

[11] Michael R. Garey, David S. Johnson, and Ravi Sethi. "The complexity of flowshop and jobshop scheduling". In: *Mathematics of Operations Research* 1.2 (1976), pp. 117–129.

[12] Oliver Holthaus and Chandrasekharan Rajendran. "Efficient dispatching rules for scheduling in a job shop". In: *International Journal of Production Economics* 48.1 (1997), pp. 87–105.

[13] Zangir Iklassov et al. "Learning to generalize dispatching rules on the job shop scheduling problem". In: *arXiv preprint arXiv:2206.04423* (2022).

[14] Diederik P. Kingma and Jimmy Ba. "Adam: A method for stochastic optimization". In: *arXiv preprint arXiv:1412.6980* (2014).

[15] Kun Lei et al. "A multi-action deep reinforcement learning framework for flexible job-shop scheduling problem". In: *Expert Systems with Applications* 205 (2022), p. 117717.

[16] Yang Lei et al. “Hybrid intelligence for dynamic job-shop scheduling with deep reinforcement learning and attention mechanism”. In: *arXiv preprint arXiv:2201.00548* (2022).

[17] Dongchen Li et al. “Revisiting experience replay in non-stationary environments”. In: *Adaptive and Learning Agents (ALA) Workshop at AAMAS*. 2021.

[18] Renke Liu, Rajesh Piplani, and Carlos Toro. “Deep reinforcement learning for dynamic scheduling of a flexible job shop”. In: *International Journal of Production Research* 60.13 (2022), pp. 4049–4069.

[19] Chao Lu et al. “Curriculum reinforcement learning algorithm for flexible job shop scheduling problems”. In: *Journal of National University of Defense Technology* 47.2 (2025), pp. 49–59.

[20] Volodymyr Mnih et al. “Human-level control through deep reinforcement learning”. In: *Nature* 518.7540 (2015), pp. 529–533.

[21] Sanmit Narvekar et al. “Curriculum learning for reinforcement learning domains: A framework and survey”. In: *Journal of Machine Learning Research* 21.181 (2020), pp. 1–50.

[22] S. S. Panwalkar and Wafik Iskander. “A survey of scheduling rules”. In: *Operations Research* 25.1 (1977), pp. 45–61.

[23] Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. 5th ed. Springer, 2016.

[24] Constantin Waubert de Puiseau, Hasan Tercan, and Tobias Meisen. “Curriculum learning in job shop scheduling using reinforcement learning”. In: *4th Conference on Production Systems and Logistics (CPSL)*. arXiv:2305.10192. 2023.

[25] Antonin Raffin et al. “Stable-Baselines3: Reliable reinforcement learning implementations”. In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8.

[26] John Schulman et al. “High-dimensional continuous control using generalized advantage estimation”. In: *arXiv preprint arXiv:1506.02438* (2016).

[27] John Schulman et al. “Proximal Policy Optimization Algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017).

[28] Igor G. Smit et al. “Graph neural networks for job shop scheduling problems: A survey”. In: *Computers & Operations Research* (2025). In press.

[29] Wen Song et al. “Flexible job shop scheduling via graph neural network and deep reinforcement learning”. In: *IEEE Transactions on Industrial Informatics* 19.2 (2023), pp. 1600–1610.

[30] Richard S. Sutton and Andrew G. Barto. “Reinforcement learning: An introduction”. In: (2018).

[31] Mark Towers et al. *Gymnasium*. Farama Foundation. `https://gymnasium.farama.org`. 2023.

[32] Runqing Wang et al. “Flexible job shop scheduling via dual attention network-based reinforcement learning”. In: *IEEE Transactions on Neural Networks and Learning Systems* (2024). arXiv:2305.05119.

[33] Annie Xie, James Harrison, and Chelsea Finn. “Deep reinforcement learning amidst continual structured non-stationarity”. In: *Proceedings of the 38th International Conference on Machine Learning (ICML)*. 2021, pp. 11393–11403.

[34] Cong Zhang et al. “Learning to dispatch for job shop scheduling via deep reinforcement learning”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020.

All source code, training scripts, and environment definitions are available in the project repository. Training was conducted for 7M steps (curriculum) using Stable-Baselines3 2.7.1 and PyTorch 2.11.0.